OpenSearch

Recherche, Analyse et Visualisation de Données

David THIBAU – 2026

david.thibau@gmail.com

Agenda — 1 / 2

1

Introduction

Historique, comparaison Elasticsearch, composants, architecture du cluster

Document & Bulk API

2

Mapping des données

Types, analyzers, dynamic templates, ISM

3

Ingestion de données

Alternative, SDK, Data Prepper, Kafka Connect

4

Requêtes et agrégations

Query DSL, filtres, full-text, agrégations, pagination, optimisation

suite →

Agenda — 2 / 2

5

Visualisation — OpenSearch Dashboards

Dashboards, visualisations, DQL, PPL, filtres interactifs, partage

6

Administration et sécurité

Nœuds, HA, snapshots, certificats, rôles et utilisateurs

7

Supervision et optimisation

Performance Analyzer, scaling, gestion du stockage, maintenance

📌 *Chaque module inclut des ateliers pratiques. Durée indicative : 3 jours.*

Introduction

01 Offre
02 Composants principaux
03 Architecture cluster
04 Indexation et documents

Offre

Naissance · Gouvernance · Positionnement · ElasticSearch

Naissance d'OpenSearch — Chronologie

2010	Création d'Elasticsearch
2015	Elastic Stack (ELK → Elastic Stack)
Jan 2021	Elastic change la licence vers SSPL/Elastic License 2.0
Avr 2021	AWS fork ES 7.10 → OpenSearch 1.0
Sep 2021	OpenSearch 1.0 GA — Projet open source
2023	OpenSearch Foundation (Linux Foundation)
2024-25	OpenSearch 2.x — roadmap indépendante

Pourquoi le fork ? — La rupture de licence

Elastic — Elastic License 2.0 / SSPL	OpenSearch — Apache License 2.0
Restrictions sur l'utilisation dans les services cloud (AWS, GCP…)	Licence Apache 2.0 garantie pour toujours (engagement fondation)
Interdit de proposer ES/Kibana en tant que service managé sans accord commercial	Sécurité, monitoring, alertes, ML : 100% gratuits et intégrés nativement
Source disponible mais pas OSI Open Source	Contributions ouvertes — gouvernance via OpenSearch Foundation
Fonctionnalités sécurité/monitoring devenues payantes (X-Pack)	Compatible cloud et on-premise sans restriction d'usage

Gouvernance et Écosystème

Indépendance

Depuis 2024, le projet est hébergé par la OpenSearch Software Foundation au sein de la Linux Foundation.

Neutralité & Contributeurs

Plus de 400 organisations contributrices : SAP, Uber, Capital One…
Pas de dépendance exclusive à un seul fournisseur (Vendor Lock-in).

Chiffres clés (2026)

Plus d'1 milliard de téléchargements cumulés. Licence Apache 2.0 immuable. Communauté active et croissante.

Écosystème de Plugins

Sécurité native, alertes, gestion de l'indexation (ISM), anomaly detection, observabilité, k-NN vectoriel.

Positionnement Technologique

OpenSearch : au-delà de la simple recherche full-text

Observabilité

Analyse de logs, traces (OpenTelemetry native) et métriques en temps réel.

IA Générative & Vecteurs

Moteur de recherche vectorielle k-NN pour RAG (Retrieval-Augmented Generation) et LLM.

Sécurité & SIEM

Détection d'anomalies par Machine Learning et SIEM open-source intégré.

Performance

Optimisation continue des agrégations et stockage découplé (Searchable Snapshots) réduisant le volume des données chaudes.

Similarités avec Elasticsearch

Apache Lucene

Les deux moteurs reposent sur Apache Lucene comme bibliothèque de recherche sous-jacente. Même index inversé, même scoring BM25.

Concepts fondamentaux identiques

Index, document, shard, replica — les concepts de base sont rigoureusement identiques entre les deux plateformes.

Query DSL hérité (v7.10)

Le Query DSL est quasi-identique, héritage direct de la version 7.10.2. Les requêtes, filtres et agrégations de base fonctionnent sans modification.

API REST compatible

CRUD (Index/Get/Delete), Search API, Bulk API, Cat API — les endpoints REST principaux sont pleinement compatibles.

Ingest pipelines

Les pipelines d'ingestion avec processors (grok, date, rename…) fonctionnent de manière identique dans les deux solutions.

Comparaison — Licences & Gouvernance

Critère	Elasticsearch	OpenSearch
Licence	Elastic License 2.0 / SSPL	Apache License 2.0
Propriétaire	Elastic N.V. (société commerciale)	OpenSearch Foundation (Linux Foundation)
Open Source OSI	Non	Oui ✔
Offre Cloud	Elastic Cloud (payant)	Amazon OpenSearch Service, self-managed
Sécurité gratuite	Non (avant 8.x)	Oui — intégrée nativement
Monitoring gratuit	Non (payant X-Pack)	Oui — Performance Analyzer inclus
Machine Learning	Payant (Platinum)	Oui — ML intégré gratuit (depuis 2.x)

Comparaison — APIs & Compatibilité

API / Fonctionnalité	Elasticsearch 8.x	OpenSearch 2.x
Query DSL	Identique	Identique ✔
REST API CRUD	Identique	Identique ✔
Index API	Identique	Identique ✔
Agrégations	Identique	Identique ✔
Sécurité (TLS, RBAC)	X-Pack payant / inclus 8.x	Plugin Security — gratuit
Index State Mgmt (ISM)	ILM (Elastic)	ISM — équivalent libre
Alerting	Watcher (payant avant)	Alerting Plugin — gratuit
SQL	Payant (X-Pack)	SQL Plugin — gratuit
k-NN / Vectoriel	ESANN (payant)	k-NN Plugin — gratuit

Versions & Compatibilité — Référence

Point de départ commun : Elasticsearch 7.10.2 (open source, jan 2021)

Elasticsearch

ES 7.10 (jan 2021)

Dernier commit open source

ES 7.11+ (fév 2021)

Passage à Elastic License 2.0

ES 8.x (2022+)

Sécurité activée par défaut, API breaking changes

OpenSearch

OS 1.0 (juil 2021)

Fork ES 7.10, sécurité intégrée gratuite

OS 1.x (2021-2022)

Stabilisation, plugins, Dashboards

OS 2.x (2022+)

Améliorations ML, ISM, alerting, k-NN, SQL

Composants principaux

OpenSearch · Dashboards · Plugins

OpenSearch — Caractéristiques principales

Distribué & Scalable

Architecture massivement distribuée basée sur Lucene. Scaling horizontal natif — adapté au Big Data.

Haute disponibilité

Réplication des shards sur plusieurs nœuds. Tolérance aux pannes sans interruption de service.

Sécurité native gratuite

TLS, authentification, RBAC, audit logs — inclus sans licence supplémentaire.

Query DSL complet

Recherche exacte, full-text, vectorielle (k-NN), géolocalisation, agrégations complexes.

API REST cohérente

API documentaire, index, cluster, cat, search — compatible avec les clients ES existants.

Plugins riches & gratuits

Alerting, SQL, Anomaly Detection, ML, ISM, Observability — tout gratuit et open source.

OpenSearch — Le moteur (Engine)

Le cœur : indexation + requêtage

1

Réception

Documents JSON reçus via REST API (Index API, Bulk API). Validation et routage vers le shard primaire approprié.

2

Analyse textuelle

Analyzers / Tokenizers : découpage du texte, lowercasing, stemming, stop words → construction de l'index inversé (Lucene).

3

Recherche & Scoring

Query DSL : full-text, exact match, vectoriel. Tri, scoring BM25/TF-IDF, agrégations statistiques et hiérarchiques.

OpenSearch Dashboards — Equivalent de Kibana

Fork de Kibana 7.10 · Licence Apache 2.0 · Développement actif indépendant

1	**DevTools Console**	*Interface équivalente à la Dev Console Kibana pour tester les API REST OpenSearch directement dans le navigateur.*
2	**Discover**	*Exploration des données brutes — filtres, champs, timeline. Équivalent fonctionnel du Discover Kibana.*
3	**Visualizations & Dashboards**	*Création de graphiques (barres, camemberts, cartes, métriques) et assemblage en tableaux de bord interactifs.*
4	**Index Management (ISM)**	*Gestion des politiques de cycle de vie des index — équivalent de l'ILM Elasticsearch mais gratuit.*
5	**Security UI**	*Gestion des utilisateurs, rôles, et permissions directement depuis l'interface — sans plugin payant.*
6	**Alerting & Anomaly Detection**	*Configuration d'alertes et détection d'anomalies ML via une interface dédiée.*

Plugins — Extensibilité par design

OpenSearch est extensible par design. Les plugins ajoutent des fonctionnalités (mappings, scripts, sécurité, observabilité…).

Security

TLS, authentification, RBAC, audit logs

Alerting

Notifications sur seuils et conditions

Anomaly Detection

Détection d'anomalies par Machine Learning

k-NN

Recherche vectorielle pour IA/RAG/LLM

ISM

Index State Management : gestion cycle de vie

SQL

Requêtes SQL sur les index OpenSearch

Observabilité

Logs, traces, métriques (OTel native)

ML Commons

Framework ML intégré gratuit

⚠ *Installation et configuration dépendantes du packaging (self-hosted vs managé). Attention aux compatibilités de versions.*

Architecture du cluster

Nœuds · Shards · Réplicas · Indexation et recherche

Cluster — Organisation et rôles des nœuds

Un cluster = plusieurs nœuds (VM / containers) qui coopèrent pour distribuer stockage et requêtes.

cluster-manager	Gestion du cluster state, élections, routage des shards. Anciennement « master ».
data	Stockage des shards. Exécute les opérations d'indexation et de recherche localement.
ingest	Pré-traitement des documents avant indexation (pipelines ingest, enrichissement).
coordinating	Reçoit les requêtes client, les distribue aux shards concernés et agrège les résultats.

Shards et Réplicas

Primary Shards

Un index est découpé en N primary shards (partitionnement horizontal).

Chaque shard = instance Lucene indépendante.

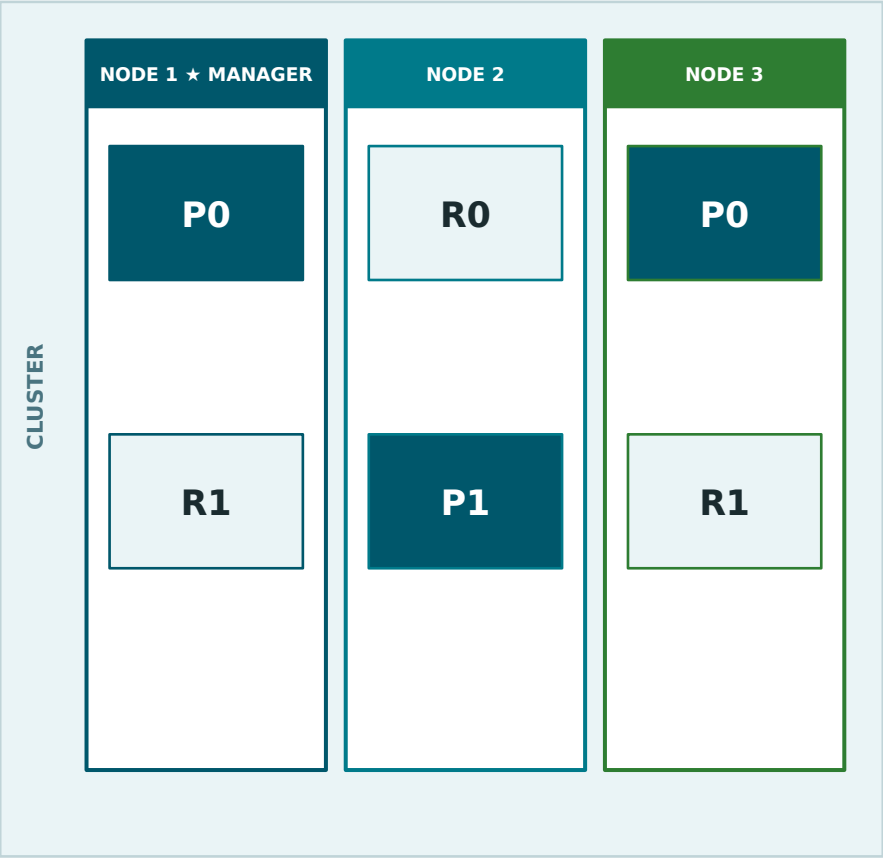
Distribués sur les nœuds du cluster pour paralléliser stockage et requêtes.

⚠ Nombre de shards fixé à la création — décision structurante.

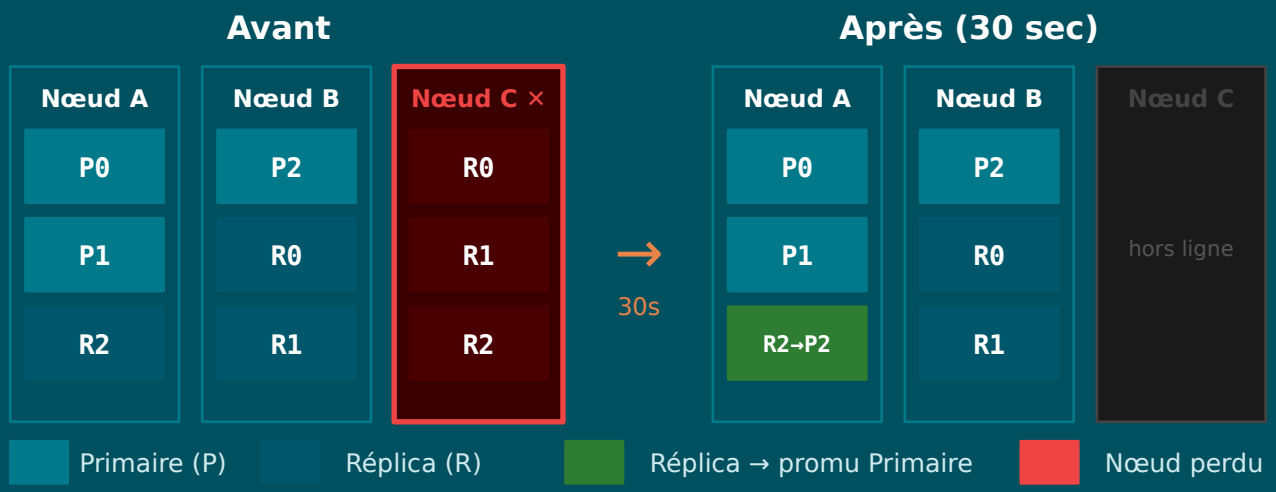
Réplicas

Copies des primary shards → Haute Disponibilité + parallélisme lecture.

Réplication = coût stockage + amélioration perf lecture.



Gestion des shards — Perte d'un nœud



Ce qui se passe

1. Heartbeat timeout (~30s) : le cluster_manager détecte la perte du nœud.
2. Les réplicas des shards primaires perdus sont promus immédiatement en nouveaux primaires.
3. Les réplicas manquants (ceux qui étaient sur le nœud perdu) passent UNASSIGNED.
4. Après le délai delayed_timeout (1 min), la recovery démarre : copie depuis les primaires.
5. Cluster revient GREEN quand tous les réplicas sont reconstruits.

⚙ index.unassigned.node_left.delayed_timeout (défaut 1m) — délai avant recovery pour laisser le nœud redémarrer. Réduire à 0 pour recovery immédiate.

Gestion des shards — Arrivée d'un nouveau nœud

Avant (2 nœuds)

Nœud A	Nœud B
P0	P2
P1	P3
R2	R0
R3	R1

→ **Nœud C** rejoint

Après rebalancing

Nœud A	Nœud B	Nœud C ★	
P0	P2	P3	RELOCATING
P1	R0	R3	RELOCATING
R2	R1	R2	

Cluster : GREEN tout le long

États de shard à surveiller

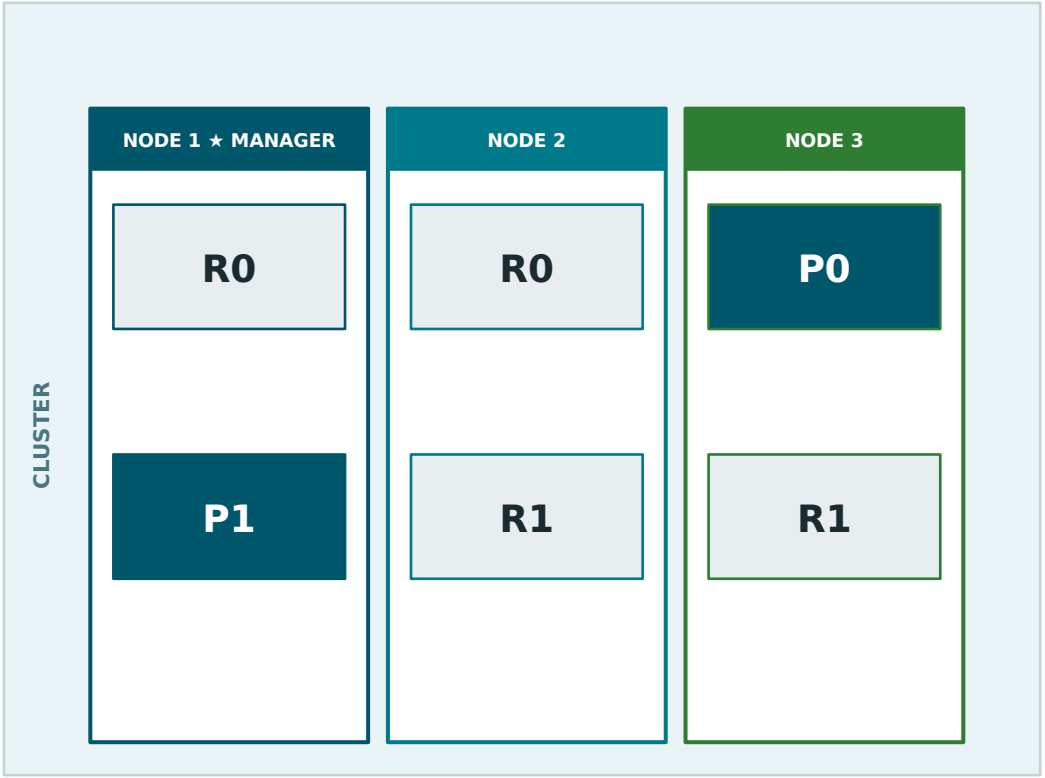
STARTED	Shard actif et opérationnel.	`_cat/shards?v&h=index,shard,prirep,state,node`
INITIALIZING	En cours de copie/récupération depuis un autre nœud.	`_cat/recovery?v&active_only=true`
RELOCATING	En cours de déplacement vers un nouveau nœud.	`_cat/shards?v  (chercher RELOCATING)`
UNASSIGNED	Non alloué — cluster en attente ou en erreur.	`_cluster/allocation/explain`

⚙ cluster.routing.rebalance.enable (all | primaries | none) — contrôle si OpenSearch rééquilibre automatiquement à l'arrivée d'un nœud.

Cycle d'indexation : du JSON au segment Lucene

1. **Réception** — Bulk / Index API reçoit le document JSON. Routage vers le shard primaire responsable.
2. **Ingest Pipeline** — Optionnel : transformation, enrichissement, grok, date parsing, géolocalisation…
3. **Analyse textuelle** — Tokenisation, lowercasing, stemming, suppression stop words → termes indexés.
4. **Écriture Lucene** — Écriture dans segments Lucene en mémoire (buffer). Flush → segment sur disque.
5. **Refresh** — Refresh (défaut 1s) : nouveau segment devient visible → document « searchable ».

Séquence d'une mise à jour (write path)



① Le client envoie la requête vers le nœud 1 (coordinating).

② Nœud 1 déduit le shard cible (hash de _id) et transfère vers nœud 3 (shard P0 primaire).

③ Nœud 3 exécute sur le shard primaire, puis réplique en parallèle vers nœud 1 et nœud 2.

④ Après confirmation des réplicas, nœud 3 répond à nœud 1 qui répond au client.

```
Formule de routing :   shard = hash(routing_key) % nombre_de_shards_primaires
```

Requête : coordination & exécution distribuée

Deux phases principales : QUERY puis FETCH

Phase QUERY

1. Le nœud recevant la requête devient coordinateur.
2. Il diffuse la requête à UNE copie (primaire ou réplique) de CHAQUE shard.
3. Chaque shard exécute la recherche localement et construit une file de priorité (IDs + scores).
4. Les résultats partiels remontent au coordinateur.

Phase FETCH

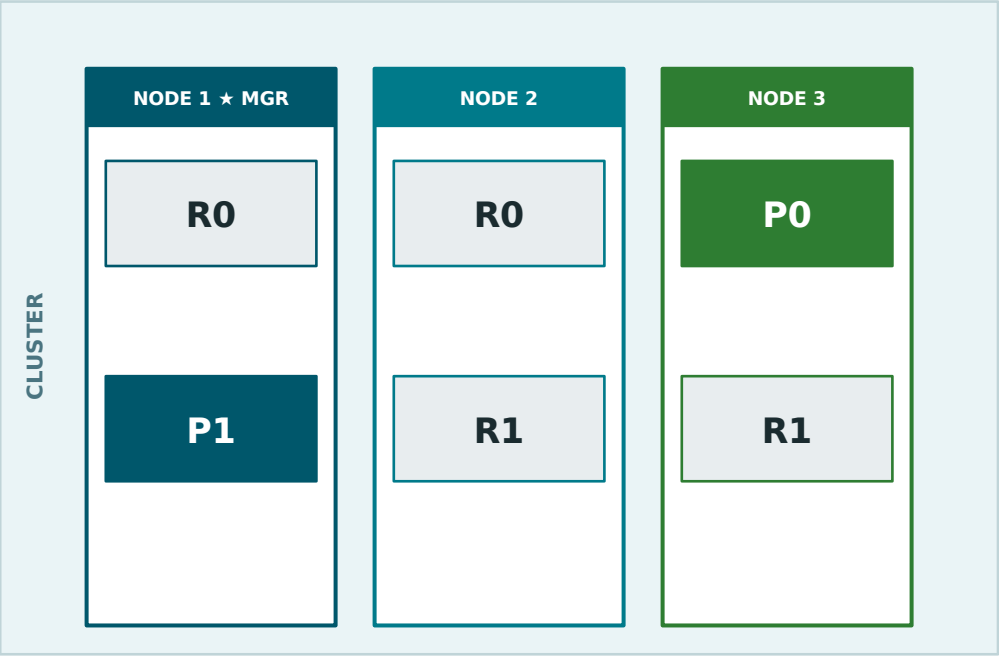
1. Le coordinateur agrège tous les résultats partiels.
2. Tri global, application de la pagination (from/size).
3. Récupération des documents complets via leurs IDs (_source).
4. Renvoi de la réponse finale au client.

⚠ Impact perf : latence = shard le plus lent. Caches, filtres, tri, agrégations → coût CPU/mémoire.

Phase Query — Détail d'exécution

Durant la 1ère phase, le coordinateur diffuse la requête à une copie (primaire ou réplique) de tous les shards.

Chaque shard exécute la recherche et construit une file à priorité des documents qui matchent.



① Le coordinateur envoie la requête en parallèle vers une copie de chaque shard.

② Chaque shard exécute localement : scoring, tri, construction de la priority queue.

③ Chaque shard retourne ses top-N résultats (IDs + scores) au coordinateur.

Phase Fetch — Récupération et retour des résultats

Le coordinateur agrège les résultats des différents shards en fonction des paramètres de pagination et de tri.

Il récupère ensuite les documents via leur _id et retourne la réponse au client.

① Agrégation des résultats partiels

Le coordinateur collecte les priority queues de tous les shards et réalise un tri global.

② Pagination

Application des paramètres from / size. Sélection des N documents à retourner.

③ Multi-get (_mget)

Le coordinateur interroge les shards qui hébergent les documents sélectionnés pour récupérer leur _source.

④ Réponse au client

Assemblage de la réponse finale : hits, _score, _source, agrégations. Retour HTTP 200 au client.

Installation & Distributions

Tarball / ZIP

```
Téléchargement depuis opensearch.org
amazon : tar -xzf opensearch-2.x.x.tar.gz
```

Packages DEB / RPM

```
apt install opensearch
yum install opensearch
```

Docker

```
docker pull
opensearchproject/opensearch:latest
docker-compose (recommandé pour cluster)
```

Kubernetes / Helm

```
Helm chart officiel OpenSearch
Opensearch Operator (k8s natif)
```

✏ **Atelier 1 — Installation cluster OpenSearch et premier démarrage**

Indexation et Documents

Structure d'un document JSON · Document API (CRUD) · Routing & Distribution

Structure d'un document OpenSearch

Champs typés

Chaque champ a un type détecté automatiquement ou défini dans le mapping : text, keyword, integer, date, boolean, geo_point, object…

Imbrication

Un champ peut contenir un objet JSON imbriqué (object) ou un tableau de valeurs.

Méta-données

_index, _id, _version, _seq_no, _primary_term — associées automatiquement à chaque document.

```
POST /clients/_doc
{
  "name": "John Smith",  // keyword/text
  "age": 42,        // integer
  "actif": true,     // boolean
  "inscription": "2024-01-15", // date
  "adresse": {

// object
    "ville": "Paris",
    "cp": "75001"
  },
  "tags": ["vip", "premium"]  // tableau
}
```

⚠ **OpenSearch : le champ _type (_doc) est déprécié depuis la v2 — un index = un type de document**

Document API — Opérations CRUD

POST	**Créer (ID auto)**	`POST /mon-index/_doc`	ID généré automatiquement par OpenSearch
PUT	**Créer / Remplacer**	`PUT /mon-index/_doc/{id}`	Crée ou remplace le document entier — _version incrémenté
GET	**Récupérer**	`GET /mon-index/_doc/{id}`	Retourne le document et ses méta-données (_index, _id, _version, _source)
DELETE	**Supprimer**	`DELETE /mon-index/_doc/{id}`	Suppression asynchrone — répliques supprimées en batch

Mise à jour partielle — _update & Scripts Painless

Principe — documents immuables

- Les documents OpenSearch sont immuables
- Une mise à jour = récupération + fusion + réindexation
- ***_update*** fusionne les champs fournis avec les champs existants
- Seuls les champs modifiés sont envoyés dans la requête

Exemples

```
# Mise à jour partielle
POST /clients/_update/1
{ "doc": { "tags": ["vip"], "score": 99 } }
```

```
# Script Painless — incrément
POST /clients/_update/1
{

"script": {

"source": "ctx._source.score += params.inc",

"params": { "inc": 10 }
  }
}
```

Bulk API — Opérations en lot

Mode batch d'OpenSearch

Regroupe plusieurs opérations (create, index, update, delete) en une seule requête HTTP — réduit drastiquement la charge réseau.

Traitement indépendant

Chaque opération est traitée séparément. L'échec d'une n'affecte pas les autres. ⚠ La Bulk API ne supporte pas les transactions.

Format NDJSON

```
{ action: { _index, _id } }  ← ligne action/metadata
{ corps du document }        ← ligne corps (sauf delete)
{ action: { ... } }
{ corps ... }
```

- Chaque ligne se termine par \n
- Pas de formatage JSON multi-lignes — tout sur une seule ligne
- delete n'a pas de ligne corps

Exemple

```
POST /mon-index/_bulk
{ "delete":  { "_id": "1" } }
{ "create":  { "_id": "2" } }
{ "titre": "Premier article" }
{ "index":   { "_id": "3" } }
{ "titre": "Deuxième article" }
{ "update":  { "_id": "2" } }
{ "doc": { "titre": "Article mis à jour" } }
```

✏ **Atelier 2.1 — Document API : CRUD sur un index OpenSearch**

API de recherche — Search Lite & Query DSL

Search Lite (URI Search)

Paramètres passés directement dans l'URL via ?q=. Syntaxe Lucene Query String.
⚠ À éviter en production : risque de sécurité, pas de contrôle fin des droits.

```
GET /mon-index/_search?q=titre:opensearch
```

Query DSL (corps JSON)

Langage riche de requête passé dans le corps de la requête. C'est l'approche recommandée en production pour la précision, la lisibilité et la sécurité.

```
GET /mon-index/_search
{ "query": { "match": { "titre":
"opensearch" } } }
```

Champs de la réponse

hits.total	Nombre de documents correspondants
hits.hits	Tableau des 10 premiers résultats (configurable via size)
_score	Score de pertinence — tri par défaut
took	Durée d'exécution en ms
_shards	Nombre de shards ayant participé

Paramètres de recherche — Pagination & Options

Pagination

```
# Page 1 (défaut : 10 résultats)
GET /mon-index/_search?size=10&from=0

# Page 2
GET /mon-index/_search?size=10&from=10

# Ou dans le corps DSL
{ "from": 20, "size": 10,
  "query": { ... } }
```

⚠ Deep pagination (from > 10 000) : éviter — utiliser l'API scroll ou search_after pour les grands volumes.

Options courantes

?pretty	Formate le JSON pour la lisibilité
?_source=false	Exclut le document source de la réponse
?_source_includes	Sélectionne des champs spécifiques
?sort=champ:desc	Tri personnalisé (ex: date:desc)
?timeout=5s	Timeout — retourne les hits trouvés à expiration
?explain=true	Explication du score de pertinence
?filter_path	Filtre les champs de la réponse

```
GET /_search  # tous index     GET /index1/_search  # index unique     GET /idx1,idx2/_search    # multi-index
```

Pagination avancée — Scroll API & search_after

⚠ *Deep pagination : coût mémoire exponentiel — OpenSearch doit charger et trier TOUS les résultats précédents.*

Scroll API

USAGE Export massif, traitement batch (ETL, réindexation)

1 Initialiser le contexte :

```
GET /mon-index/_search?scroll=1m
{ "size": 1000, "query": { "match_all": {} } }
```

2 Paginer avec le _scroll_id :

```
POST /_search/scroll
{ "scroll": "1m", "scroll_id": "<id>" }
```

Contexte maintenu en mémoire côté serveur · Déconseillé pour requêtes temps réel

search_after ✓ Recommandé

USAGE Pagination profonde temps réel, UI infinite scroll, APIs

1 Première requête (avec sort obligatoire) :

```
GET /mon-index/_search
{ "size": 1000,
  "sort": [ { "@timestamp": "asc" }, { "_id": "asc" } ] }
```

2 Page suivante — utiliser sort du dernier hit :

```
{ "search_after": ["2024-01-15T10:00:00", "abc123"],
  "sort": [ { "@timestamp": "asc" }, { "_id": "asc" } ] }
```

Stateless — aucun contexte serveur · Stable avec index actif · PIT (Point In Time) recommandé

✏ Atelier 2.2 — Search Lite

Mapping

01 Types de données

02 Contrôle du mapping

03 Gabarits d’index

04 Datastream et ILM

05 Alias et migration

Types de données

Index inversé – Text vs keyword – Objets embarqués - Analyseurs

Index inversé

Afin d'accélérer les recherches, OpenSearch utilise une structure de données nommée **index inversé**. Cela consiste en une liste de mots uniques où chaque terme est associé aux documents dans lesquels il apparaît.

Principe :
1. Le texte est analysé → tokenisé
2. Chaque token est ajouté à l'index
3. La recherche consulte l'index inversé
   (pas les documents directement)

→ *Recherche en O(1) quelle que soit la taille du corpus.*

Index inversé — exemple

Terme (token)	Documents
`pizza`	3, 5
`paris`	1, 10
`lucene`	2, 3
`opensearch`	2, 4
`tomorrow`	1, 2, 4, 10
`caffè`	3, 5
`big`	6
`brown`	6
`fox`	6
`jump`	6
`the`	1,2,4,5,6,8,9

💡 *Les champs keyword utilisent doc_values (index colonne) — non l'index inversé. Optimisé pour le tri, l'agrégation, et le filtrage exact.*

Types de données — Types simples

Texte

- text — full-text analysé
- keyword — valeur exacte

Numériques

- byte, short, integer, long
- float, double
- token_count

Dates & Booléens

- date (ISO8601, epoch ms)
- boolean (true/false)
- binary (base64)

Intervalles

- integer_range
- float_range / long_range
- date_range

Géolocalisation

- geo_point (lat/lon)
- geo_shape (polygones)

Spéciaux

- ip (IPv4 ou IPv6)
- percolator (requête JSON)

⚠ *Le type string n'existe plus dans OpenSearch. Utiliser text (recherche full-text) ou keyword (valeur exacte, tri, agrégation) selon l'usage.*

keyword vs text — Valeur exacte ou Full-text

keyword — Valeur exacte

Stockée telle quelle — aucune analyse.
Opérateurs : term, terms, filter

Usages :
- **IDs, codes, emails, statuts**
- **Tri et agrégations**

Sensible à la casse :
"Foo" ≠ "foo" ≠ "FOO"

```
GET /clients/_search
{
  "query": {
    "term": { "status": "actif" }
  }
}
```

text — Full-text analysé

Analysée et découpée en tokens.
Opérateurs : match, match_phrase, query_string

Usages :
- **Textes, articles, commentaires**
- **Recherche en langage naturel**

"Paris" → token: "paris"

```
GET /articles/_search
{
  "query": {
    "match": { "titre": "moteur recherche" }
  }
}
```

Types complexes — array, object, nested

[ ] Tableaux (array)

Pas de type de mapping spécial.
Chaque champ accepte 0, 1 ou n valeurs.
Les valeurs doivent être du même type.
Un champ null ≡ tableau vide.

```
{ "tags": ["search", "nosql",
"api"] }
```

{ } Objets imbriqués (object)

Structure JSON embarquée.
OpenSearch aplatit les objets en interne.
Accès par chemin pointé : adresse.ville
Mapping auto sous properties.

```
{ "adresse": {
  "ville": "Paris",
  "cp": "75001"
}}
```

⊞ Nested

Tableau de structures embarquées.
Chaque élément = document Lucene caché.
Permet des requêtes corrélées sur les sous-objets.

```
{ "tags": [
  {"nom":"Alice","role":"admin"},
  {"nom":"Bob","role":"user"}
]}
```

⚠ *Nested génère beaucoup de documents Lucene internes — à utiliser avec parcimonie sur les champs à très haut volume.*

Mapping des objets embarqués

OpenSearch détecte dynamiquement les champs objets et les mappe comme **`object`**. Chaque sous-champ est listé sous **`properties`**.

Document JSON

```
POST /social/_doc
{
  "tweet": "Je cherche avec OpenSearch",
  "user": {
    "id": "u001",
    "age": 28,
    "name": {
      "first": "Alice",
      "last":  "Dupont"
    }
  }
}
```

Mapping généré automatiquement

```
GET /social/_mapping
{
  "properties": {
    "tweet": { "type": "text" },
    "user": {
      "type": "object",
      "properties": {
        "id":  { "type": "text" },
        "age": { "type": "long" },
        "name": {
          "type": "object",
          "properties": {
            "first": { "type": "text" },
            "last":  { "type": "text" }
          }
        }
      }
    }
  }
}
```

💡 *En interne, OpenSearch aplatit user.name.first — il n'y a pas de JSON imbriqué stocké, mais des champs à chemin pointé (dot notation).*

Analyseurs — Principe et pipeline

Les analyseurs transforment un champ **text** en un flux de tokens qui constituent l'**index inversé**. Ils s'appliquent à l'indexation ET à la recherche.

1

Character Filters

Pré-traitement du texte
(strip HTML, remplacements)

→

2

Tokenizer

Découpe en tokens
(standard, whitespace, ngram)

→

3

Token Filters

Normalise les tokens
(lowercase, stop words, stem)

Exemple avec l'analyseur french

Original

```
"Les données sont
analysées"
```

→

Tokenizer

```
"Les" "données"
"sont" "analysées"
```

→

Lowercase

```
"les" "données"
"sont" "analysées"
```

→

Stop words

```
"données"
"analysées"
```

→

Stemming

```
"donn"
"analys"
```

Analyseurs intégrés

Analyseur	Description	Usage typique
`standard`	Sépare en mots, supprime la ponctuation, passe en minuscule. (défaut)	*Usage général, multilingue*
`french`	Stemming français, élision (l', d'…), stop words français.	*Textes en français*
`english`	Stemming anglais (Snowball), stop words anglais.	*Textes en anglais*
`whitespace`	Découpe uniquement sur les espaces. Préserve la casse et la ponctuation.	*Codes avec séparateurs*
`keyword`	Aucune analyse. La valeur entière est un seul token.	*Champs non analysés*
`simple`	Découpe sur les non-lettres, passe en minuscule.	*Texte simple*
`stop`	Comme simple + suppression des mots courants (stop words).	*Textes avec mots courants*
`pattern`	Découpe selon une expression régulière (défaut : \W+).	*Logs, CSV, formats structurés*

💡 *L'analyseur à l'indexation DOIT être identique à celui de la recherche — sinon les tokens ne correspondent pas et les résultats sont vides.*

Test des analyseurs — API _analyze

OpenSearch expose l'API ***_analyze*** pour tester le comportement d'un analyseur sur un texte, sans indexer de document.

Requête

```
# Sur l'index (avec son analyzer déclaré)
GET /mon-index/_analyze
{
  "analyzer": "french",
  "text": "Les données sont analysées"
}

# Sur un analyzer global
GET /_analyze
{
  "analyzer": "standard",
  "text": "Text to analyze"
}
```

Réponse

```
{
  "tokens": [
    {
      "token":        "text",
      "start_offset": 0,
      "end_offset":   4,
      "type":         "<ALPHANUM>",
      "position":     1
    },
    {
      "token":        "analyz",
      "start_offset": 8,
      "end_offset":   15,
      "type":         "<ALPHANUM>",
      "position":     3
    }
  ]
}
```

💡 *start_offset / end_offset / position permettent de comprendre le highlighting et les recherches de proximité (phrase queries).*

✏ Atelier 3.1 — Test des analyseurs

Contrôle du mapping

Dynamic mapping – Mapping explicite – Dynamic templates

Dynamic Mapping — true / false / strict

La propriété **`dynamic`** contrôle le comportement lors de l'apparition d'un nouveau champ non connu dans un document indexé :

✅ dynamic: true

Ajout automatique (défaut)

Tout nouveau champ est automatiquement détecté et ajouté au mapping.

Risque de mapping explosion si les documents ont des structures très variables.

⏭️ dynamic: false

Ignoré silencieusement

Les nouveaux champs sont acceptés dans _source mais ne sont pas indexés ni interrogeables.

Utile pour les champs volatils à stocker sans indexer.

🚫 dynamic: "strict"

Exception levée

Tout nouveau champ inconnu déclenche une erreur 400 à l'indexation.

Recommandé en production pour garantir la cohérence du schéma.

```
PUT /my_index { "mappings": { "dynamic": "strict", "properties": { "title": {"type":"text"}, "stash":
{"type":"object","dynamic":true}}}}
```

Comportement par défaut du mapping

Lorsqu'OpenSearch détecte un nouveau champ **`string`**, il le configure automatiquement comme **2 champs** :

Champ text

- Analyseur standard appliqué
- Utilisable pour les recherches full-text (match)
- Non utilisable pour le tri ou les agrégations

+

Sous-champ keyword

- Valeur exacte, non analysée
- Tri, agrégations, filtres exacts
- Accès via `nom.keyword`

```
# "city" génère : city (text) + city.keyword (keyword)
GET /clients/_search
{
  "query": { "match":  { "city": "paris" }},                ← full-text (text)
  "aggs":  { "villes": { "terms": { "field": "city.keyword" }}}  ← agrégation (keyword)
}
```

⚠ *Pour éviter ce double mapping automatique, spécifier le mapping explicitement à la création de l'index.*

Règles de détection par défaut

Type JSON détecté	Type OpenSearch assigné
`null`	Champ ignoré — non ajouté au mapping
`true / false`	boolean
`Entier (42)`	long
`Décimal (3.14)`	float
`{ ... }`	object (sous-champs dans properties)
`[ ... ]`	Dépend du type de la première valeur non-nulle du tableau
`"2024-01-15"`	date (si le format correspond aux formats configurés)
`"42" / "3.14"`	long ou float (si numeric_detection: true)
`"Paris"`	text (analyseur standard) + sous-champ keyword (.raw)

💡 *Configurer dynamic_date_formats et numeric_detection: true pour adapter la détection automatique à votre format de données.*

✏ Atelier 3.2 — Mapping dynamique

Création d'index — settings, mappings, aliases

Lors de la création d'un index, 3 blocs peuvent être configurés :

aliases
Noms secondaires pour l'index

settings
Shards, répliques, analyzers...

mappings
Types de champs, analyzers

```
PUT /mon_index
{
  "aliases": {
    "logs": {},
    "logs_recent": { "filter": { "range": { "date": { "gte": "now-7d" }}}}
  },
  "settings": {
    "number_of_shards":   3, "number_of_replicas": 1, "refresh_interval":   "1s",
    "analysis": { analyzer": {
        "mon_fr": { "type": "custom", "tokenizer": "standard", "filter": ["french"] }
      } } },
  "mappings": {
    "dynamic": "strict",
    "properties": {
      "titre":     { "type": "text",    "analyzer": "mon_fr" },
      "categorie": { "type": "keyword" },
      "date":      { "type": "date",    "format": "yyyy-MM-dd" },
      "score":     { "type": "float",   "index": false }
    } } }
```

⚠ *number_of_shards est immuable après création. Anticiper la volumétrie ! number_of_replicas peut être modifié à tout moment.*

Mapping personnalisé

Un mapping personnalisé permet de contrôler précisément la structure et le comportement de chaque champ :

🎯 Type de champ

Forcer geo_point, ip, date avec format custom, nested...

🔍 Analyseur spécifique

Assigner french, english ou un analyseur custom à un champ text.

🔑 Exact vs Full-text

Décider si un champ est keyword (exact) ou text (analysé).

📅 Format de date

Spécifier le format attendu : yyyy-MM-dd, epoch_millis, etc.

🚫 Désactiver l'index

"index": false pour stocker sans permettre la recherche.

⚙ Multi-fields

Un même champ avec plusieurs types (text + keyword + analyzer dédié).

⚠ *Le mapping d'un champ existant ne peut **PAS** être modifié. Seul l'ajout de nouveaux champs est possible via PUT /index/_mapping.*

Spécification du mapping

On peut spécifier le mapping lors de la création d'un index, ou lors de l'ajout d'un nouveau champ dans un index existant.

Création de l'index

```
PUT /gb
{
  "mappings": {
    "properties": {
      "tweet": {
        "type":     "text",
        "analyzer": "french"
      },
      "date": {
        "type":   "date",
        "format": "yyyy-MM-dd"
      },
      "name": {
        "type": "keyword"
      },
      "user_id": {
        "type": "long"
      }
    }
  }
}
```

Ajout sur index existant

```
PUT /gb/_mapping
{
  "properties": {
    "tag": {
      "type":  "keyword"
    },
    "score": {
      "type":  "float",
      "index": false
    }
  }
}
```

⚠ **Champ déjà indexé = immuable**
Pour modifier un type : créer un nouvel index + `POST /_reindex`, puis basculer l'alias.

✏ **Atelier 3.3 — Mapping explicite**

Gabarits dynamiques — dynamic_templates

Les **`dynamic_templates`** définissent des règles de mapping personnalisées appliquées automatiquement aux nouveaux champs détectés. Les règles sont traitées dans l'ordre — la première qui correspond est appliquée.

Critères de matching

`match_mapping_type`	Type JSON détecté
`match`	Pattern sur le nom
`unmatch`	Exclusion de nom
`match_pattern`	Regex sur le nom
`path_match`	Chemin complet
`path_unmatch`	Exclusion de chemin

Exemple

```
PUT my_index
{
  "mappings": {
    "dynamic_templates": [
      {
        "integers": {
          "match_mapping_type": "long",
          "mapping": { "type": "integer" }
        }
      },
      {
        "strings": {
          "match_mapping_type": "string",
          "mapping": {
            "type": "text",
            "fields": {
              "raw": { "type": "keyword", "ignore_above": 256 }
            }
          }
        }
      }
    ]
  }
}
```

💡 *Les gabarits d'index (index templates) définissent en général des dynamic_templates réutilisables sur tous les index du pattern.*

✏ Atelier 3.4 — Dynamic templates

Gabarits d’index

Index template – Component template

Gabarits d'index — Index Templates

Un index template définit automatiquement les **settings, mappings et aliases** appliqués à chaque nouvel index dont le nom correspond au pattern défini.

Nouveau doc vers logs-2025-03 → **Pattern match logs-*** → **Template appliqué automatiquement** → **Index créé avec le bon mapping**

Création d'un index template

```
PUT _index_template/template_logs
{
  "index_patterns": ["logs-*"],
  "priority": 500,
  "template": {
    "settings": { "number_of_shards": 1, "number_of_replicas": 1, "refresh_interval": "5s" },
    "mappings": {
      "dynamic": "strict",
      "properties": {
        "@timestamp": { "type": "date" },
        "level":      { "type": "keyword" },
        "message":    { "type": "text", "analyzer": "standard" },
        "service":    { "type": "keyword" }
      }
    },
    "aliases": { "all_logs": {} }
  }
}
```

Component Templates — Gabarits de composants

Depuis OpenSearch 1.x (ES 7.8+), les index templates sont construits à partir de **component templates** — des blocs réutilisables indépendants.

🧩 Component template

- Bloc de configuration réutilisable
- Définit mappings, settings ou aliases
- Ne s'applique PAS directement à un index
- Peut être partagé entre plusieurs index templates

📐 Index template (composé)

- Référence un ou plusieurs component templates
- Peut ajouter ses propres settings/mappings
- S'applique aux index dont le nom match le pattern
- Priorité configurable (en cas de conflit)

🗂️ Gabarits prédéfinis OpenSearch

- logs-*-*
- metrics-*-*
- synthetics-*-*
- profiling-*

💡 *Les component templates n'ont pas d'effet tant qu'ils ne sont pas référencés par un index template via composed_of.*

Création de component templates

On crée des component templates indépendants, puis on les assemble dans un index template via composed_of :

Component 1 — champ @timestamp

```
PUT _component_template/component_timestamp
{
  "template": {
    "mappings": {
      "properties": {
        "@timestamp": { "type": "date" }
      }
    }
  }
}
```

Component 2 — champ runtime calculé

```
PUT _component_template/component_runtime
{
  "template": {
    "mappings": {
      "runtime": {
        "jour_semaine": {
          "type": "keyword",
          "script": { "source":
            "emit(doc['@timestamp'].value
              .dayOfWeekEnum.getDisplayName(
                TextStyle.FULL, Locale.FRENCH))"
          }
        }
      }
    }
  }
}
```

Assemblage dans index template

Index template composé — assemblage des component templates

```
PUT _index_template/template_logs
{
  "index_patterns": ["logs-*"],
  "priority": 500,
  "composed_of": ["component_timestamp", "component_runtime"],    ← assemblage des composants
  "template": {
    "settings": { "number_of_shards": 1, "number_of_replicas": 1 },
    "mappings": { "properties": { "level": { "type": "keyword" }, "message": { "type": "text" }}},
    "aliases": { "all_logs": {} }
  },
  "version": 1,
  "_meta": { "description": "Template logs OpenSearch" }
}
```

💡 *Les mappings du template composé sont fusionnés avec ceux des component templates. En cas de conflit, le template composé l'emporte.*

Datastreams et ILM

Roll over - Policy

Datastreams — Introduction

Les **datastreams** permettent de stocker des séries d'événements horodatés (logs, métriques, traces) via un nom unique, tout en gérant automatiquement plusieurs index en arrière-plan.

@timestamp obligatoire

Chaque document doit contenir un champ @timestamp de type date ou date_nanos. OpenSearch refuse l'indexation sans ce champ.

Index de sauvegarde masqués

Un datastream repose sur un ou plusieurs index générés automatiquement (.ds-logs-2025.03-000001). Ils sont masqués mais interrogeables.

Write index unique

À tout moment, un seul index du datastream est l'index d'écriture. Les nouveaux documents y sont acheminés automatiquement.

Lié à un index template

Un datastream est obligatoirement associé à un index template avec un pattern correspondant. C'est le template qui définit le mapping.

```
PUT _data_stream/logs-app-prod          ← Créer un datastream (le template doit déjà exister)
GET _data_stream/logs-app-prod          ← Inspecter un datastream
DELETE _data_stream/logs-app-prod       ← Supprimer (supprime aussi les index de sauvegarde)
POST /logs-app-prod/_doc                ← Indexer (toujours vers le write index)
  { "@timestamp": "2025-03-12T10:00:00Z", "level": "INFO", "message": "Démarrage OK" }
```

⚠ *Un datastream est en append-only : les mises à jour et suppressions de documents nécessitent de cibler directement l'index de sauvegarde.*

Datastreams — Write index & Rollover

Architecture d'un datastream

Indexing request

→

→

Data stream : logs-app-prod

.ds-logs-app-prod 2025.01-000001	.ds-logs-app-prod 2025.02-000002	**.ds-logs-app-prod 2025.03-000003**
read only	**read only**	✍ **write index**

🔄 Rollover — Rotation de l'index d'écriture

⚙ Automatique (ILM)

L'ILM déclenche un rollover quand l'index atteint l'âge ou la taille configurés.

📏 Conditions

max_age: 30d, max_size: 50gb, max_docs: 10000000 — configurable dans la policy ILM.

🖐 Manuel

POST /logs-app-prod/_rollover — crée un nouvel index et le désigne comme write index.

ILM — Index Lifecycle Management

L'**ILM (Index Lifecycle Management)** automatise les actions de gestion du cycle de vie des index. Il est associé à un index template pour s'appliquer automatiquement.

🔥

HOT

Index actif
Écriture + lecture
Rollover configuré

→

🌤️

WARM

Plus d'écriture
Encore interrogé
Shrink possible

→

❄️

COLD

Rarement interrogé
Requêtes plus lentes
Stockage économique

→

🗑️

DELETE

Index supprimé
après min_age
configurable

Actions disponibles par phase

`Rollover`	Créer un nouvel index quand l'actuel atteint l'âge/taille	*HOT*
`Shrink`	Réduire le nombre de shards primaires	*WARM*
`Force merge`	Réduire les segments, libérer l'espace des docs supprimés	*WARM/COLD*
`Freeze`	Rendre read-only, minimiser l'empreinte mémoire	*COLD*
`Delete`	Supprimer définitivement l'index	*DELETE*

ILM — Création d'une policy

Une policy ILM définit les phases et les conditions de transition. Elle est associée au datastream via l'index template.

```
PUT _plugins/_ism/policies/logs_policy
{
  "policy": {
    "description": "Politique logs OpenSearch",
    "phases": {
      "hot": {
        "actions": {
          "rollover": {
            "max_size":  "50gb",
            "max_age":   "30d",
            "max_docs":  10000000
          } } },
      "warm": {
        "min_age": "30d",
        "actions": {
          "shrink": { "num_shards": 1 },
          "forcemerge": { "max_num_segments": 1 }
        } },
      "cold": {
        "min_age": "60d",
        "actions": { "freeze": {} }
      },
      "delete": {
        "min_age": "90d",
        "actions": { "delete": {} }
      }
    }
  }
}
```

💡 Points clés

ISM vs ILM

OpenSearch utilise le plugin ISM (Index State Management) — équivalent de l'ILM Elasticsearch. L'API est _plugins/_ism.

Rollover conditions

max_size, max_age et max_docs sont combinables — le rollover se déclenche dès qu'une condition est atteinte.

min_age

Délai minimum après rollover avant de passer à la phase suivante. Hot n'a pas de min_age (actif immédiatement).

Lien template

Associer la policy au template via index.plugins.ism.policy_id dans les settings.

Mise en place d'un datastream — Étapes

La mise en place d'un datastream s'effectue en plusieurs étapes, dans cet ordre :

1 **Créer la policy ISM (ILM)**

Définir les phases hot/warm/cold/delete et les conditions de transition (max_age, max_size...).

```
PUT _plugins/_ism/policies/ma_policy { ... }
```

2 **Créer les component templates**

Définir les blocs réutilisables : mapping du @timestamp, des champs métier, des settings standards.

```
PUT _component_template/mon_composant { ... }
```

3 **Créer l'index template**

Assembler les component templates + définir le pattern, la priorité et lier la policy ISM.

```
PUT _index_template/mon_template { "index_patterns":["logs-*"],
"composed_of":[...] }
```

4 **Créer le datastream**

Créer explicitement le datastream — le premier index de sauvegarde est généré automatiquement.

```
PUT _data_stream/logs-app-prod
```

5 **Sécuriser le datastream**

Définir les rôles et permissions via OpenSearch Security (lecture, écriture, gestion).

```
PUT _plugins/_security/api/roles/logs_role { ... }
```

☞ **Atelier 3.5 — Index Templates et ISM**

Alias et migration

Alias – Reindex – Bonnes pratiques

Alias

Un alias est un **nom secondaire** pour un groupe d'index ou de datastreams. Toutes les API OpenSearch acceptent un alias à la place d'un index.

■ Zero-downtime reindex

Pointer l'alias vers le nouvel index après réindexation, sans changer le code applicatif.

🔖 Abstraction multi-index

logs-* → alias logs : écrire/lire sur tous les index via un seul nom.

🔧 Alias filtré

Exposer une vue partielle d'un index (ex : uniquement les docs d'un tenant).

✏ Alias d'écriture

Désigner un seul index actif en écriture parmi plusieurs (is_write_index: true).

```
POST _aliases {
  "actions": [
    { "add":    { "index": "logs-2025-01", "alias": "logs_write", "is_write_index": true },}
    { "add":    { "index": "logs-2025-*",  "alias": "logs_recent", "filter": { "range": { "@timestamp": { "gte": "now-30d" }}}}}
    { "remove": { "index": "logs-2024-12", "alias": "logs" }}  ]
}
```

📌 *Bonne pratique : les applications ne référencent jamais un index directement — toujours via un alias. Facilite les migrations et rolling upgrades.*

Migration – Pourquoi migrer un mapping ?

Un mapping OpenSearch n'est pas librement modifiable après création. Certains changements imposent une réindexation complète des données.

✅ Possible sans reindex

- ✓ Ajouter un nouveau champ (dynamique ou explicite)
- ✓ Ajouter un sous-champ (.keyword, .ngram…)
- ✓ Modifier index: false sur un champ text
- ✓ Changer ignore_malformed, ignore_above
- ✓ Ajouter copy_to vers un nouveau champ
- ✓ Étendre un analyzer existant (puis _close/_open)

✗ Nécessite un reindex

- ✗ Changer le type d'un champ (text→keyword, long→double…)
- ✗ Changer l'analyzer d'un champ déjà indexé
- ✗ Modifier le nombre de shards primaires
- ✗ Supprimer un champ du mapping
- ✗ Changer la stratégie de date detection
- ✗ Renommer un champ (reindex + script Painless)

⚠ Les types de champs existants sont gravés dans l'index dès le premier document indexé. Anticiper le mapping (mode strict + templates) est toujours préférable à le corriger a posteriori.

Migration – L'opération Reindex

_reindex copie les documents d'un index source vers un index de destination. L'index destination doit être créé au préalable avec le mapping cible.

Reindex simple et pilotage

```
# 1. Créer l'index cible avec le nouveau mapping
PUT /logs-v2
{
  "settings": { "number_of_shards": 4 },
  "mappings": {
    "properties": {
      "@timestamp":  { "type": "date" },
      "level":       { "type": "keyword" },
      "http_status": { "type": "integer" }
    } } }

# 2. Reindex asynchrone (recommandé)
POST /_reindex?wait_for_completion=false
{
  "conflicts": "proceed",
  "source": { "index": "logs-v1", "size": 1000 },
  "dest":   { "index": "logs-v2", "op_type": "create" }
}
# → { "task": "nodeId:taskId" }

# 3. Suivre la progression
GET /_tasks/nodeId:taskId
GET /_tasks?actions=*reindex&detailed=true
```

```
POST /_reindex  # Avec modification de champ
{
  "source": { "index": "logs-v1" },
  "dest":   { "index": "logs-v2" },
  "script": {
    "lang": "painless",
    "source": """
   // Renommer un champ
   ctx._source.level = ctx._source.remove('log_level');
   if (ctx._source.http_status != null) {
     ctx._source.status = Integer.parseInt(
          ctx._source.status); }
   ctx._source.remove('deprecated_field');
    """ } }
```

```
POST /_reindex # Depuis un cluster distant
{
  "source": {
    "remote": {
      "host": "https://old-cluster:9200",
      "username": "admin", "password": "pass" },
    "index": "logs-v1", "size": 500
  },
  "dest": { "index": "logs-v2" }
}
# opensearch.yml : reindex.remote.whitelist
```

Migration – Stratégie zero downtime avec les alias

En combinant alias et reindex, il est possible de migrer un index sans interruption de service ni perte de données. C'est la stratégie de référence en production.

1. **L'application utilise l'alias 'logs' → logs-v1**
   Écriture et lecture passent par l'alias, jamais directement par le nom d'index
2. **Créer logs-v2 avec le nouveau mapping cible**
   PUT /logs-v2 — l'alias 'logs' pointe encore sur v1, aucune interruption
3. **Reindex asynchrone logs-v1 → logs-v2**
   wait_for_completion=false — la prod continue d'écrire sur v1 pendant le reindex
4. **Delta reindex : rattraper les nouveaux docs**
   Reindex ciblant @timestamp ≥ date de début — combler l'écart avant basculement
5. **Basculement atomique de l'alias**
   POST /_aliases — remove logs→v1 + add logs→v2 en une seule opération
6. **Supprimer logs-v1 après validation**
   Conserver 24–48h pour permettre un rollback, puis DELETE /logs-v1

Commandes clés

```
POST /_aliases # Alias initial avec is_write_index
{
  "actions": [{
    "add": { "index": "logs-v1", "alias": "logs",
"is_write_index": true } }]
}

POST /_reindex # Delta reindex
{
  "source": {
    "index": "logs-v1",
    "query": { "range": {
        "@timestamp": { "gte": "2024-01-15T10:00:00Z" }
      }  } },
  "dest": { "index": "logs-v2" }
}


POST /_aliases # Basculement ATOMIQUE
{
  "actions": [
 { "remove": { "index": "logs-v1", "alias": "logs" } },
 { "add": { "index": "logs-v2", "alias": "logs",
                  "is_write_index": true } }] }
```

Migration – Bonnes pratiques et pièges à éviter

Une migration bien planifiée est transparente pour les utilisateurs. Les problèmes surviennent presque toujours par manque d'anticipation ou de tests.

✅ Recommandations

Toujours utiliser des alias en production

Ne jamais exposer directement un nom d'index aux applications — rend tout basculement sans downtime possible.

Versionner les index (v1, v2, v3…)

Conserver l'ancien index pendant la migration et 24–48h après pour permettre un rollback rapide.

Tester les requêtes critiques sur v2 avant basculement

Un champ manqué ou mal typé dans le nouveau mapping = régression silencieuse en production.

Monitorer la vitesse du reindex

GET /_tasks?actions=*reindex&detailed=true → docs/s, estimation durée restante, taux d'erreur.

Planifier le delta reindex

Calculer la fenêtre temporelle de rattrapage avant de couper. Un @timestamp range query suffit souvent.

⛔ Pièges à éviter

Ne pas tester les performances du nouveau mapping

Un keyword remplaçant un text non-analyzed peut multiplier la mémoire fielddata par 10.

Oublier de mettre à jour les index templates

Si un template crée automatiquement des index (logs-*), le mettre à jour AVANT de migrer.

Reindex avec size par défaut (1000) sur de gros docs

Adapter size en fonction de la taille des documents pour éviter les timeouts ou sous-optimisations.

Supprimer l'ancien index trop vite

Attendre 24–48h minimum pour valider et permettre un rollback. Le stockage est moins cher qu'une indisponibilité.

Ignorer les pipelines d'ingestion actifs

Un Ingest Pipeline actif transforme les docs à l'écriture — reproduire ces transformations dans le script Painless du reindex.

✏ Atelier 3.6 — Réindexation et migration de mapping

Ingestion

01 Alternatives
02 SDK Clients
03 Data Prepper
04 KafkaConnect

Ingestion – Vue d'ensemble des alternatives

OpenSearch peut recevoir des données depuis de nombreuses sources. Le choix dépend du volume, de la latence, de la transformation requise et de l'existant.

Bulk API
Fondamental

REST native · POST /_bulk · Lots JSON · Base de tous les agents

→ Toujours — brique de base

SDK Clients
Programmatique

Java, Python, Go, JS · Wrappers Bulk API · Retry auto + backoff

→ Ingestion depuis appli métier

Data Prepper
Natif OpenSearch

Pipeline ETL OpenSearch · Sources : Kafka, S3, HTTP, OTel, Grok

→ Nouveaux projets, OTel, logs

Kafka Connect
Streaming

Sink connector OpenSearch · At-least-once · Batching configurable

→ Architecture événementielle

Logstash
Écosystème ELK

200+ plugins · Compatible via opensearch-output · Migrations ES

→ Réutiliser pipelines existants

Beats / Agent
Edge

Filebeat, Metricbeat · Faible empreinte · Modules préconfigurés

→ Collecte edge : serveurs, VMs

💡 *Data Prepper et Logstash utilisent tous deux la Bulk API en coulisses — comprendre la Bulk API est la clé de toutes les optimisations de débit.*

SDK Clients

Intro – Exemple Java – Bulk – Ingestion massive - Reprise sur erreur

SDK Clients – Vue d'ensemble

OpenSearch fournit des clients officiels pour les langages les plus courants. Tous encapsulent la Bulk API et fournissent gestion des connexions, sérialisation et retry automatique.

opensearch-java

Java / Kotlin v3.x

Client typé · Fluent Builder API · Compatible OpenSearch 2.x+

opensearch-py

Python v2.x

helpers.bulk · parallel_bulk · async (aiohttp) · pandas integration

opensearch-go

Go v2.x

Typage strict · Faible empreinte · Idiomatic Go API

opensearch-js

Node.js v3.x

Async/await · Streams · Compatible Browser (bundle)

opensearch-ruby

Ruby v3.x

ActiveRecord style · Rails integration

opensearch-rs

Rust v0.2

Community · Async tokio · Zero-copy deserialization

⚠ *Vérifier la compatibilité : les clients Java/Python 2.x et antérieurs ciblaient Elasticsearch. Utiliser les clients officiels opensearch-* pour les versions 2.x+ d'OpenSearch.*

SDK Python – opensearch-py · Bulk & parallel_bulk

opensearch-py fournit ***helpers.bulk()*** et ***helpers.parallel_bulk()*** qui simplifient l'ingestion volumineuse avec gestion automatique des erreurs et du backoff.

Installation & connexion

```
pip install opensearch-py

from opensearchpy import OpenSearch, helpers

client = OpenSearch(
    hosts=[{'host': 'localhost', 'port': 9200}],
    http_compress=True,     # gzip: -40% réseau
    retry_on_timeout=True,
    max_retries=3,
    maxsize=16,             # pool connexions
)
```

helpers.bulk()

Simple

Ingestion séquentielle mono-thread. Gère les erreurs partielles. Idéal pour < 100k docs.

helpers.parallel_bulk()

Haute perf

N threads simultanés. Générateur asynchrone. Recommandé pour ingestion volumineuse.

Async (aiohttp)

Async

from opensearchpy import AsyncOpenSearch · await client.bulk(). Idéal FastAPI / asyncio.

Streaming bulk

Memory

Le pattern generator évite de charger toutes les données en mémoire — critique pour TB de données.

SDK Python – Exemple parallel_bulk

Generator pattern + parallel_bulk

```
def generate_actions(records):
    for rec in records:
        yield { # streaming
            '_index': 'catalogue',
            '_source': rec,
            # '_id': rec['id']  # optionnel
        }

ok, failed = 0, 0
# Envoie les documents en //
for success, info in helpers.parallel_bulk(client,
    generate_actions(data),
    thread_count=4,       # threads parallèles
    chunk_size=500,       # docs par batch
    max_chunk_bytes=10*1024*1024,  # 10 Mo max
    raise_on_error=False,
):
    if success: ok += 1
    else: failed += 1; print(info)
print(f'ok={ok} failed={failed}')
```

SDK Java – OpenSearch Java Client 3.x

Le client Java officiel utilise une API Fluent Builder typée. Il gère nativement le pool de connexions, la sérialisation Jackson/JSON-b et le retry avec backoff exponentiel.

Dépendance + configuration du client

```
<!-- build.gradle -->
Implementation org.opensearch.client:opensearch-java:3.1.0

RestClientTransport transport = new RestClientTransport(
    RestClient.builder(new HttpHost("localhost",9200)).build(),
    new JacksonJsonpMapper());
OpenSearchClient client = new OpenSearchClient(transport);
```

BulkRequest

Simple

Construction manuelle du batch. Contrôle total. Synchrone.

BulkIngester

Production

Buffer auto + flush périodique + listener erreurs. Recommandé pour applications long-running.

Thread-safety

⚠ Important

Le client est thread-safe. Utiliser une seule instance partagée dans Spring / Quarkus.

SSL / Auth

Sécurité

SSLContext + BasicCredentials configurés dans le RestClient. Gestion via Keystore ou variables env.

SDK Java – OpenSearch Java Client 3.x

Le client Java officiel utilise une API Fluent Builder typée. Il gère nativement le pool de connexions, la sérialisation Jackson/JSON-b et le retry avec backoff exponentiel.

Dépendance + configuration du client

```
<!-- build.gradle -->
Implementation org.opensearch.client:opensearch-java:3.1.0

RestClientTransport transport = new RestClientTransport(
    RestClient.builder(new HttpHost("localhost",9200)).build(),
    new JacksonJsonpMapper());
OpenSearchClient client = new OpenSearchClient(transport);
```

BulkRequest

Simple

Construction manuelle du batch. Contrôle total. Synchrone. Retry sur transport HTTP, les autres erreurs sont à gérer

BulkIngester

Production

Buffer auto + flush périodique + listener erreurs. Recommandé pour applications long-running. Retry sur erreur transitoire de type 429

Thread-safety

⚠ Important

Le client est thread-safe. Utiliser une seule instance partagée dans Spring / Quarkus.

SSL / Auth

Sécurité

SSLContext + BasicCredentials configurés dans le RestClient. Gestion via Keystore ou variables env.

SDK Java – API d'indexation de base · CRUD documentaire

Chaque opération retourne un objet Response — pas de Map<String,Object> à manipuler.

Indexer un document (index)

```
// Créer ou remplacer — ID auto-généré
IndexResponse resp = client.index(i -> i
    .index("produits-java")
    .document(monProduit)
);
String id     = resp.id();
String result = resp.result().jsonValue(); // "created"
long version  = resp.version();
// Avec ID explicite :
client.index(i -> i.index("produits-java")
    .id("PROD-001").document(monProduit));
```

Récupérer un document (get)

```
GetResponse<Produit> resp = client.get(g -> g
    .index("produits-java").id("PROD-001"),
    Produit.class);
if (resp.found()) {
    Produit p = resp.source(); // deserialisé
}
```

💡 *client.exists() -> boolean | client.mget() -> multi-doc | toutes les ops levent IOException*

SDK Java – API d'indexation de base · CRUD documentaire (2)

Mise a jour partielle (update)

```
// Via doc: (champs fournis seulement)
client.update(u -> u.index("produits-java")
    .id("PROD-001")
    .doc(Map.of("stock", 99, "actif", false)),
    Produit.class);

// Via script Painless
client.update(u -> u.index("produits-java").id("PROD-001")
    .script(sc -> sc.inline(i -> i
        .source("ctx._source.stock += params.n")
        .params("n", JsonData.of(5)))), Produit.class);
```

Supprimer (delete)

```
DeleteResponse dr = client.delete(d -> d
    .index("produits-java").id("PROD-001"));
// dr.result() -> "deleted" ou "not_found"
```

Créer l'index avec mapping explicite

```
client.indices().create(c -> c.index("produits-java")
    .settings(s -> s.numberOfShards("3")
                    .numberOfReplicas("2"))
    .mappings(m -> m
        .properties("nom",   p -> p.text(t ->
t.analyzer("french")))
        .properties("prix",  p -> p.float_(f -> f))
        .properties("actif", p -> p.boolean_(b -> b))));
```

> *Appeler createIndex() au demarrage —*
> *verifier l'existence avec indices().exists().*

SDK Java – BulkRequest & BulkIngester · Ingestion par lots

BulkRequest pour les batchs ponctuels avec controle total.
BulkIngester pour les applications long-running avec flush automatique par taille, nombre ou intervalle.

BulkRequest – batch manuel

```
BulkRequest.Builder br = new BulkRequest.Builder();

for (Produit p : produits) {
    br.operations(op -> op
        .index(i -> i.index(INDEX).document(p)));
}

BulkResponse resp = client.bulk(br.build());

// Analyser item par item — HTTP 200 meme si erreurs !
if (resp.errors()) {
    resp.items().forEach(item -> {
        if (item.error() != null)
            log.error("[{}] {} : {}", item.status(),
                item.id(), item.error().reason());
    });
}
long tookMs = resp.took();
```

💡 *1 appel HTTP par appel. Idéal ETL/migration.*
Taille recommandée : 500-5000 docs ou 5-15 Mo.
resp.errors() est le seul indicateur d'echec partiel.

BulkIngester – flush automatique

```
// Bean @Service singleton — thread-safe
try (BulkIngester<Produit> ing = BulkIngester.of(b -> b
    .client(client)
    .maxOperations(500)         // flush a 500 docs
    .maxSize(10 * 1024 * 1024L) // ou 10 Mo
    .flushInterval(5, TimeUnit.SECONDS)
    .listener(new BulkListener<Produit>() {
        public void afterBulk(long execId,
                BulkRequest req, List<Produit> ctx,
                BulkResponse resp) {
            resp.items().stream()
                .filter(i -> i.error() != null)
                .forEach(i -> log.error(
                    i.error().reason()));
        }
        public void afterBulk(long execId,
                BulkRequest req, List<Produit> ctx,
                Throwable failure) {
            log.error("Bulk failed", failure);
        }
    })
)) {
produits.forEach(p -> ing.add(op -> op.index(...))); }
```

💡 *une commande = un doc, evenements continus.*
Bean singleton.

SDK Java – Ingestion massive · Optimisations d'index

Avant une ingestion volumineuse, desactiver refresh et replicas reduit le trafic inter-noeuds et le cout des segments Lucene — a restaurer imperativement apres.

1 AVANT *Parametres ingestion* -> **2 INGESTION** *BulkIngester* -> **3 APRES** *Restaurer + refresh*

setIngestionMode() + orchestration

```
public void setIngestionMode(boolean on)
        throws IOException {
    client.indices().putSettings(s -> s
        .index(INDEX)
        .settings(cfg -> cfg
            .refreshInterval(t -> t.time(
                on ? "-1" : "1s"))
            .numberOfReplicas(
                String.valueOf(on ? 0 : 2)))
    );
    log.info("Mode ingestion {} : refresh={} replicas={}",
        on, on?"-1":"1s", on?0:2);
}

public void forceRefresh() throws IOException {
    client.indices().refresh(r -> r.index(INDEX));
}

// -- Orchestration complete --
setIngestionMode(true);
var stats = ingesterBulk(generateProduits(50_000));
setIngestionMode(false);
forceRefresh();
```

Impact de chaque parametre

`refresh_interval: "-1"` — **+++ debit**

Supprime la creation continue de segments Lucene. Aucune visibilite en temps reel — acceptable pour un chargement initial ou une migration.

`number_of_replicas: 0` — **+++ debit**

Elimine la replication inter-noeuds. Sur 3 noeuds + 2 replicas, chaque doc est ecrit 3x — passer a 0 supprime 2/3 du trafic reseau.

`translog.durability: async` — **++ debit**

Flush du translog toutes les 30s au lieu de chaque ecriture. Risque de perte sur crash — acceptable pour donnees rejouables.

`flush_threshold_size: 2gb` — **+ debit**

Evite les micro-flushes qui generent de petits segments couteux a merger par Lucene.

SDK Java – Gestion des erreurs & patterns de robustesse

La Bulk API retourne HTTP 200 meme en cas d'erreurs partielles. Il faut analyser chaque item et rejouer uniquement les erreurs transitoires avec backoff exponentiel.

Codes d'erreur Bulk a gerer

429 **Too Many Requests - throttling**

-> Retry avec backoff exponentiel + jitter

400 **Mapping error - type incompatible**

-> Corriger le document, ne pas retry

503 **Service Unavailable - noeud surchargé**

-> Retry apres pause (backoff)

409 **Conflict - create sur ID existant**

-> Utiliser index au lieu de create

Regle d'or : ne jamais retry les erreurs 4xx (sauf 429) - elles ne passeront pas mieux au 2e essai.

SDK Java – Retry - Exemple

```
public BulkStats bulkWithRetry( List<Produit> docs, int maxRetries) throws IOException, InterruptedException {

    List<Produit> pending = new ArrayList<>(docs);
    long success = 0;

    for (int attempt = 0; attempt < maxRetries && !pending.isEmpty(); attempt++) {
        BulkRequest.Builder br = new BulkRequest.Builder();
        for (Produit p : pending)
            br.operations(op -> op.index(i -> i.index(INDEX).document(p)));

        BulkResponse resp = client.bulk(br.build());
        List<Produit> toRetry = new ArrayList<>();
        for (int k = 0; k < resp.items().size(); k++) {
            var item = resp.items().get(k);
            if (item.error() == null) { success++;
            } else if (item.status() == 429 || item.status() == 503) {
                toRetry.add(pending.get(k));
            } else {
                log.error("Non-retriable [{}]: {}", item.status(), item.error().reason());
            }
        }
        pending = toRetry;
        if (!pending.isEmpty()) {
            // Backoff exponentiel + jitter
            long wait = (1L << attempt) * 1000L + (long)(Math.random() * 500);
            Thread.sleep(wait);
        }
    }
    return new BulkStats(success, pending.size(), 0);
}
```

✏ Atelier 4.1 — Ingestion via SDK Java

Data Prepper

Architecture – Processeurs - OpenTelemetry

Ingest Pipelines — Introduction

Transformer les documents avant indexation —
sans Logstash ni Data Prepper

⚙ Processors enchaînés : *grok, date, rename, convert, enrich, geo_ip…*

📍 Exécutés par OpenSearch au moment de l'indexation, avant écriture dans le shard

🔗 Attachables à n'importe quel index via ?pipeline= ou un mapping par défaut

Ingest Pipelines — Processors principaux

grok	Parsing regex	`"pattern": ["%{IPORHOST:ip}"]`	Extrait des champs structurés depuis une chaîne de texte brute.
date	Normalisation date	`"formats": ["dd/MMM/yyyy"]`	Parse et normalise une date vers @timestamp ou un champ cible.
rename	Renommer un champ	`"field": "log_level"` `"target_field": "level"`	Renomme un champ. Alternative au script Painless en réindexation.
convert	Changer le type	`"field": "status"` `"type": "integer"`	Convertit string → integer / float / boolean / ip sans Painless.
enrich	Enrichissement	`"policy_name": "geo-enrich"`	Ajoute des champs depuis un index de référence (lookup par valeur).
geo_ip	IP → Géo	`"field": "client_ip"`	Résout une IP en coordonnées GPS, pays, ville via base GeoLite2.

Ingest Pipelines — Créer et utiliser un pipeline

JSON / OpenSearch DSL

```
# 1. Créer le pipeline
PUT /_ingest/pipeline/logs-apache
{
  "processors": [
    { "grok": {
        "field": "message",
        "patterns": ["%{IPORHOST:client_ip} %{WORD:method} %
{URIPATHPARAM:url} HTTP/%{NUMBER} %{NUMBER:status_code}"]
    }},
    { "date": {
        "field": "timestamp_raw",
        "formats": ["dd/MMM/yyyy:HH:mm:ss Z"],
        "target_field": "@timestamp"
    }},
    { "convert": { "field": "status_code", "type":
"integer" }},
    { "remove":  { "field": "message" }}
  ]
}

# 2. Indexer avec le pipeline
POST /logs-apache/_doc?pipeline=logs-apache
{ "message": "192.168.1.10 GET /api/v1 HTTP/1.1 200 1024",
  "timestamp_raw": "15/Jan/2024:10:23:45 +0100" }
```

Étape 1 : grok

Parse la ligne de log brute (format Combined Log) et crée automatiquement les champs client_ip, method, url, status_code.

Étape 2 : date

Normalise timestamp_raw vers @timestamp au format ISO 8601 — exploitable par le filtre temporel de Dashboards.

Étape 3 : convert

Convertit status_code de string en integer pour permettre les agrégations et les range queries numériques.

Étape 4 : remove

Supprime le champ message d'origine qui ne sert plus — les données structurées le remplacent.

Data Prepper – Architecture & pipeline

Data Prepper est le pipeline d'ingestion natif d'OpenSearch. Il ingère des traces OTel, des logs et des métriques via un modèle sources → buffer → processors → sinks.

pipeline YAML – logs Apache

```
# /etc/data-prepper/pipelines.yaml
log-pipeline:
  source:
    http:
      ssl: false
      port: 2021
  processor:
    - grok:
        match:
          message: ['%{COMMONAPACHELOG}']
    - date:
        from_time_received: true
        destination: "@timestamp"
    - drop_events:
        drop_when: '/status == "200"'
  sink:
    - opensearch:
        hosts: ["https://os:9200"]
        username: "admin"
        password: "${OS_PASSWORD}"
        index: "apache-logs-%{yyyy.MM.dd}"
        bulk_size: 4
        flush_timeout: 60000
```

Data Prepper vs Logstash – Comparatif

Data Prepper et Logstash résolvent le même problème mais avec des philosophies différentes. Le choix dépend du contexte et de l'existant.

Critère	Data Prepper	Logstash
Créé par	AWS / OpenSearch Foundation	Elastic
Licence	Apache 2.0 (libre)	Elastic License 2.0 (non libre)
Config	YAML déclaratif	DSL Ruby-like (plus expressif)
Plugins	~20 sources/processors natifs	200+ plugins (écosystème massif)
OTel natif	✓ OTLP gRPC/HTTP nativement	✗ Pas de support natif OTel
Conditions	drop_when (limité)	if/else/elsif complets
Enrichissement	geoip, aggregate, mutate	JDBC lookup, translate, ruby
Maturité	Jeune (2021)	Très mature (2013)

Choisir Data Prepper

- Traces OpenTelemetry → OpenSearch APM
- Nouveaux projets logs structurés JSON
- Architecture AWS / S3 / Kafka
- Simplicité opérationnelle prioritaire

Choisir Logstash

- Migration depuis Elastic Stack avec Beats
- Logique conditionnelle complexe
- Enrichissement JDBC (base de données)
- Pipelines Logstash existants à réutiliser

Data Prepper – Démarrage Docker & endpoints

Démarrage docker

```
# Démarrer Data Prepper
docker run -d \
  -v /etc/data-prepper:/usr/share/\
     data-prepper/config \
  -p 2021:2021 -p 4900:4900 \
  opensearchproject/data-prepper:latest
```

data-prepper-config.yaml minimal

```
# data-prepper/data-prepper-config.yaml
ssl: false   # Désactive HTTPS sur l'API 4900
```

Tous les endpoints disponibles

:4900	GET	/list Lister les pipelines actifs
:4900	GET	/metrics/prometheus Métriques Prometheus (débit, erreurs)
:4900	GET	/metrics/sys Métriques système JVM
:4900	POST	/shutdown Arrêt propre du processus
:2021	POST	/log/ingest Source HTTP — envoyer des logs JSON
:21890	gRPC	OTLP TraceService Traces OpenTelemetry (protocole gRPC)
:21891	HTTP	/opentelemetry.proto.* Traces OTel via HTTP (alternative)

Data Prepper – Sources et Processors disponibles

Data Prepper propose un catalogue de sources et processors couvrant logs, métriques, traces et événements. Les pipelines peuvent être chaînés pour des architectures complexes.

Sources disponibles

`http`
Réception de logs/événements via HTTP. Port configurable. TLS supporté.

`kafka`
Consommation de topics Kafka. group_id, auto.offset.reset, SASL/TLS.

`s3`
Lecture de fichiers S3 (SQS notification ou scan). CSV, JSON, Parquet.

`otel_trace`
Réception de traces OpenTelemetry (OTLP). Port 21890 (gRPC).

`file`
Lecture de fichiers locaux. Codec: json, newline_delimited.

`syslog`
Réception UDP/TCP syslog. RFC 3164 et RFC 5424 supportés.

Processors disponibles

`grok`
Extraction de champs via patterns (COMMONAPACHELOG, SYSLOGLINE…).

`date`
Parsing de dates vers @timestamp. Formats strptime configurables.

`mutate`
rename, copy, uppercase, lowercase, gsub, add_entries, remove_entries.

`geoip`
Enrichissement géographique depuis une IP. Champs lat/lon/city/country.

`aggregate`
Aggrégation d'événements (count, sum, histogram) sur fenêtre temporelle.

`drop_events`
Suppression d'événements selon condition (drop_when: expression).

`otel_trace_raw`
Transformation traces OTel → format OpenSearch APM natif.

`string_converter`
Conversion simple de champs string (upper_case, lower_case).

OpenTelemetry – Standard d'observabilité moderne

OpenTelemetry (OTel) est un standard open-source de la CNCF pour collecter, transformer et exporter des données d'observabilité — traces, métriques et logs — de manière unifiée.

🔍 Traces *Distributed Tracing*

Suit le chemin d'une requête à travers tous les services.
Chaque opération = un Span.
Ensemble de spans = une Trace.

```
User request → API Gateway → Service A
→ DB → Service B
```

📊 Métriques *Metrics*

Mesures numériques dans le temps.
Compteurs, jauges, histogrammes. CPU, latence, taux d'erreur, saturation.

```
http_requests_total{method='GET'} =
4521
```

📋 Logs *Logs*

Événements textuels structurés.
OTel standardise le format et l'enrichissement avec le contexte de trace (TraceId, SpanId).

```
{'traceId':'abc123','level':'ERROR','ms
g':'...'}
```

OTLP

OpenTelemetry Protocol — format de transport standard (gRPC ou HTTP/Protobuf).

SDK OTel

Librairies pour Java, Python, Go, JS… Instrumentation auto ou manuelle.

Collector

Agent/Gateway de collecte. Reçoit, transforme, exporte. Data Prepper en est un.

Auto-instr.

Instrumentation sans code (Java agent, Python auto-instrumentation).

💡 *OpenTelemetry est né de la fusion entre OpenCensus (Google) et OpenTracing (CNCF) en 2019.*
C'est aujourd'hui le 2e projet le plus actif de la CNCF après Kubernetes.

Data Prepper – Pipeline traces OpenTelemetry

Data Prepper est la solution de référence pour ingérer des traces OpenTelemetry dans OpenSearch. Le pipeline transforme les spans en index otel-v1-apm-* requis par Trace Analytics.

pipelines.yaml — traces OTel (docker-compose)

```
otel-trace-pipeline:
  source:
    otel_trace_source: { ssl: false, port: 21890 }
  buffer:
    bounded_blocking:
      buffer_size: 25600
      batch_size:  400
  processor:
    - otel_trace_raw_processor:
  sink:
    - pipeline: { name: raw-trace-pipeline }
    - pipeline: { name: service-map-pipeline }

raw-trace-pipeline:
  source:
    pipeline: { name: otel-trace-pipeline }
  processor:
    - otel_trace_raw_processor:
  sink:
    - opensearch:  # trace_analytics_raw: true
```

service-map-pipeline: même source → processor: service_map_stateful → sink: trace_analytics_service_map: true

Index créés automatiquement

otel-v1-apm-span-*

Spans individuels avec parentId, duration, attributs. Alimenté par trace_analytics_raw: true.

otel-v1-apm-service-map

Graphe des dépendances inter-services. Alimenté par service_map_stateful.

Configuration application cliente (OTel SDK)

```
# Variables d'environnement (toute app OTel)
OTEL_EXPORTER_OTLP_ENDPOINT=http://localhost:21890
OTEL_SERVICE_NAME=mon-service
OTEL_TRACES_EXPORTER=otlp

# Java Agent (instrumentation automatique)
java -javaagent:opentelemetry-javaagent.jar \
     -jar mon-app.jar
```

💡 *Trace Analytics dans OpenSearch Dashboards : Observability → Trace Analytics. Les index doivent être présents pour que l'interface se charge.*

✏ Atelier 4.2 — Data Prepper et OpenTelemetry

Kafka Connect

Intro – Exemple Java – Bulk – Ingestion massive - Reprise sur erreur

Kafka Connect – Architecture & concepts

Kafka Connect est un framework d'intégration distribué de l'écosystème Apache Kafka. Le connecteur OpenSearch Sink permet d'ingérer des topics Kafka vers OpenSearch en temps réel.

Producers Applications → **Apache Kafka Brokers** → **Kafka Connect Worker** → **OpenSearch Sink Connector** → **OpenSearch Cluster**

Concepts clés Kafka Connect

Connector
Plugin qui définit source ou sink. Configuré via REST API ou fichier JSON.

Task
Thread d'exécution du connector. Parallélisme configurable (tasks.max).

Worker
Processus JVM hébergeant les connectors. Mode standalone ou distribué.

Offset
Position dans le topic Kafka. Committée après ingestion réussie.

DLQ
Dead Letter Queue : topic Kafka pour les messages en erreur (irrémédiables).

SMT
Single Message Transform : transformation légère avant le sink.

```
# Standalone (dev / test)
bin/connect-standalone.sh \
  config/connect-standalone.properties \
  config/opensearch-sink.properties

# Distribué (production — REST API)
bin/connect-distributed.sh \
  config/connect-distributed.properties
# → POST http://connect-host:8083/connectors
```

Avantages pour OpenSearch

At-least-once delivery
Fiable

Offsets committés après bulk réussie. Rejeu automatique sur erreur transiente.

Backpressure native
Scalable

Kafka absorbe les pics de volume. OpenSearch Sink consomme à rythme stable.

Kafka Connect – Configuration OpenSearch Sink

Le connecteur OpenSearch Kafka Connector (opensearch-kafka-connect) est le connecteur officiel maintenu par la communauté OpenSearch.

Configuration minimale (JSON / REST API)

```
# POST http://connect:8083/connectors
{
  "name": "opensearch-sink",
  "config": {
    "connector.class":
      "io.aiven.kafka.connect.opensearch.OpensearchSinkConnector",
    "tasks.max": "2",
    "topics": "logs-app,logs-nginx",
    "connection.url": "https://os-node:9200",
    "connection.username": "admin",
    "connection.password": "${env:OS_PASS}",
    "key.ignore": "true",
    "schema.ignore": "true",
    "behavior.on.malformed.documents": "warn",
    "behavior.on.null.values": "delete",
    "batch.size": "500",
    "max.buffered.records": "20000",
    "flush.timeout.ms": "10000",
    "index.write.method": "upsert",
    "type.name": "_doc"
  }
}
```

topics → index

Mapping

Par défaut : 1 topic = 1 index (même nom). Configurer index.name pour personnaliser.

key.ignore: true

Pratique

Ignore la clé Kafka. L'_id OpenSearch est généré automatiquement.

schema.ignore: true

JSON

Ingère des messages JSON bruts sans Schema Registry. Mode le plus courant.

action

Upsert

upsert : crée ou fusionne le doc via _id
index : écrase toujours.

Contrôle du connecteur (REST API)

```
GET    /connectors                    # liste
GET    /connectors/opensearch-sink/status
PUT    /connectors/opensearch-sink/config
POST   /connectors/opensearch-sink/restart
DELETE /connectors/opensearch-sink
```

Kafka Connect – Topics to Index & Dead Letter Queue

Le mapping topics → index et la gestion des erreurs via Dead Letter Queue sont les deux points de configuration les plus importants pour un déploiement production.

Mapping topics → index personnalisé

```
# Plusieurs topics → plusieurs index
"topics": "app-logs,nginx-logs,audit-logs",

# Renommage via SMT ExtractTopic
"transforms": "topicToIndex",
"transforms.topicToIndex.type":
  "org.apache.kafka.connect.transforms.ReplaceField$Value",

# Routage dynamique via timestamp
"index.name": "logs-${topic}-%{yyyy.MM.dd}",

# Filtrage par clé de partition
"topic.index.map":
  "app-logs:catalogue,nginx-logs:nginx-access"
```

SMT – Single Message Transforms utiles

```
# Ajouter @timestamp depuis l'heure de réception
"transforms": "addTs",
"transforms.addTs.type":
  "org.apache.kafka.connect.transforms.InsertField$Value",
"transforms.addTs.timestamp.field": "@timestamp"
```

Dead Letter Queue (DLQ)

```
# Activer la DLQ
"errors.tolerance": "all",
"errors.deadletterqueue.topic.name":
  "dlq-opensearch-sink",
"errors.deadletterqueue.topic.replication.factor": "1",
"errors.deadletterqueue.context.headers.enable": "true",

# Les headers DLQ contiennent :
# kafka.connect.errors.exception.class.name
# kafka.connect.errors.exception.message
# kafka.connect.errors.record.original.value
```

Stratégie de gestion des erreurs

`errors.tolerance: none`
Arrêt immédiat du connecteur sur 1ère erreur. Mode strict. Dev/test uniquement.

`errors.tolerance: all`
Continue malgré les erreurs. Envoie vers DLQ. Mode recommandé en production.

`behavior.on.malformed.documents: warn`
Warning + skip du document malformé (mapping error). Ne bloque pas le pipeline.

✏ Atelier 4.3 — Kafka Connect

Synthèse

Ingestion – Optimisations de débit · Synthèse

Plusieurs niveaux d'optimisation peuvent être combinés pour multiplier le débit. Appliquer dans cet ordre : index settings → batch sizing → parallélisme → architecture.

1 – Paramètres index

- refresh_interval: -1 (+++ débit)
- number_of_replicas: 0 (+++ débit)
- translog.durability: async (++ débit)

2 – Batch sizing

- Taille cible : 5 – 15 Mo de payload
- 500 – 5000 docs selon taille des documents
- Mesurer avec _nodes/stats/indices

3 – Parallélisme client

- parallel_bulk thread_count: 4–8
- Pool connexions ≥ nombre de threads
- Compression gzip (http_compress=True)
- Backoff exponentiel sur 429/503

4 – Architecture

- Nœuds ingest dédiés (node.roles: [ingest])
- Kafka comme buffer absorbe les pics
- Ingest pipeline pour enrichissement léger
- Réindexer si mapping à corriger après

Batch : 5–15 Mo | Threads : 4–8 | gzip activé | Pool connexions ≥ threads | Retry backoff exponentiel | refresh_interval: -1 pendant ingestion

Ingestion – Matrice de décision

Choisir l'outil selon le cas d'usage, le volume et les contraintes de l'environnement existant.

Cas d'usage	Bulk API	SDK Python	SDK Java	Data Prepper	Kafka Connect	Logstash
Ingestion ad hoc / test	✓✓✓	✓✓	✓✓	✓	✗	✓
Application métier (API)	✗	✓✓✓	✓✓✓	✗	✗	✗
Logs structurés (fichiers)	✗	✓	✓	✓✓✓	✗	✓✓✓
Traces OpenTelemetry	✗	✗	✗	✓✓✓	✗	✗
Streaming Kafka haute perf	✗	✓	✓	✓✓	✓✓✓	✓
Migration depuis ES	✓	✓✓	✓✓	✗	✗	✓✓✓
Infra Beats existante	✗	✗	✗	✗	✗	✓✓✓
Débit maximal (batch)	✓✓✓	✓✓✓	✓✓✓	✓✓	✓✓	✓

✓✓✓ Recommandé · ✓✓ Possible · ✓ Acceptable · ✗ Déconseillé

DSL

01 Syntaxe
02 Principaux opérateurs
03 Agrégations
04 Géolocalisation

Syntaxe

Structure – Requêtes vs Filtres – Combinaison de clauses

Introduction au DSL

Pourquoi utiliser le DSL ?

- Combiner des clauses de requêtes plus facilement
- Influencer le score de pertinence
- Mettre en surbrillance des parties du résultat
- Agréger des sous-ensembles de résultats
- Retourner des suggestions à l'utilisateur

Rappel : Recherche simple vs DSL

```
GET /mon-index/_search?q=opensearch
```

→ *Recherche simple, limitée*

```
POST /mon-index/_search
{
  "query": { ... }
```

💡 Avantage OpenSearch DSL

Le DSL OpenSearch expose toute la puissance de Lucene via une interface JSON lisible et extensible.

Les requêtes DSL offrent bien plus de fonctionnalités qu'une recherche simple.

Structure d'une requête DSL

JSON / OpenSearch DSL

```
GET /_search
{
  "query": YOUR_QUERY_HERE
}
```

Le paramètre query

- Paramètre principal pour passer une clause DSL
- Expose toute la puissance du moteur Lucene
- Interface JSON lisible et structurée

DSL = Arbre syntaxique de clauses

Clauses feuilles

match, term, range
S'appliquent à un champ.
Peuvent s'exécuter seules.

Clauses composées

bool, dis_max, constant_score
Combinent d'autres clauses avec des opérateurs logiques.

Contextes

Contexte requête (score)
Contexte filtre (booléen)
Modifient le comportement.

Distinction : Requête vs Filtre

DSL exprime deux types de requêtes selon le contexte d'utilisation

🔍 Contexte REQUÊTE

Score de pertinence calculé

Chaque document reçoit un score _score

Résultats triés par pertinence décroissante

Activé par le mot-clé
`query`

Usage : recherche full-text

⚡ Contexte FILTRE

Résultat booléen uniquement

Pas de calcul de score (oui/non)

Résultats mis en cache → plus rapide

Activé par les mots-clés
`filter` ou `must_not`

Usage : valeurs exactes, dates, ranges

Activation des contextes – Exemple

JSON / OpenSearch DSL

```
GET /_search
{
  "query": {                    // ← contexte REQUÊTE
    "bool": {
      "must": [
        { "match": { "title":   "Search" }},
        { "match": { "content": "OpenSearch" }}
      ],
      "filter": [               // ← contexte FILTRE
        { "term":  { "status": "published" }},
        { "range": { "publish_date": { "gte": "2023-01-
01" }}}
      ]
    }
  }
}
```

Contexte REQUÊTE

Clauses must et match
→ calculent un score

Contexte FILTRE

Clauses filter, must_not
→ pas de score, mis en cache

💡 Bonne pratique
Utilisez filter pour les valeurs exactes : meilleure performance car les résultats sont mis en cache par OpenSearch.

Combinaison de clauses – bool

La clause bool permet de combiner plusieurs conditions avec des opérateurs logiques

JSON / OpenSearch DSL

```
  "query": {
    "bool": {
      "must":     { "match": { "tweet":
"opensearch" }},
      "must_not": { "match": { "name": "marie" }},
      "should":   { "match": { "tweet": "full text" }}
    }
  }
```

must ≡ **ET logique**

Le document DOIT correspondre.
Contribue au score de pertinence.

must_not ≡ **NOT logique**

Le document NE DOIT PAS correspondre.
Exécuté en contexte filtre (pas de score).

should ≡ **OU logique**

Le document DEVRAIT correspondre.
Améliore le score si correspondance.

Requêtes vides et pagination

JSON / OpenSearch DSL

```
# Requête vide — tous les indices
GET /_search
{
# Pagination

GET /_search

{

  "from": 30,

  "size": 10
}
```

Requête vide — tous documents

- GET /_search {}
- GET /index-2024*/_search {}
- Aucun critère → retourne tous les documents
- Utile pour inspecter un index ou tester une connexion
- Supporte les wildcards sur le nom d'index

Pagination

- from : offset de départ (défaut : 0)
- size : nombre de résultats (défaut : 10)
- ⚠ from + size > 10 000 → utiliser search_after ou scroll

Principaux opérateurs

Filtre – match - multi-match - phrase

Opérateurs dans le contexte filtre

term	Valeur exacte sur un champ	`{ "term": { "age": 26 }}`
terms	Plusieurs valeurs possibles	`{ "terms": { "tag": ["search","nosql"] }}`
range	Intervalle numérique ou de dates	`{ "range": { "age": { "gte": 20, "lt": 30 }}}`
exists	Teste la présence d'un champ	`{ "exists": { "field": "title" }}`
bool	Combine : must (ET), must_not (NOT), should (OU)	`{ "bool": { "must": [...], "should": [...] }}`

Opérateur match

La requête match est la recherche standard pour full-text ou valeurs exactes

JSON / OpenSearch DSL

```
{ "match": { "tweet": "About OpenSearch" }}

# Avec options
GET /_search
{
  "query": {
    "match": {
      "message": {
        "query":    "this is a test",
        "operator": "and"
      }
    }
  }
}
```

Comportement selon le champ

- Champ full-text → analyse la chaîne avec le même analyseur que le champ
- Champ à valeur exacte → recherche la valeur exacte

Options principales

- operator (and/or) : opérateur booléen entre les termes
- minimum_should_match : nbre minimum de clauses matchant
- analyzer : analyseur à utiliser pour la chaîne
- lenient : ignorer les erreurs de types de données

Opérateur multi_match

Exécuter la même requête sur plusieurs champs simultanément

JSON / OpenSearch DSL

```
# Recherche sur plusieurs champs
GET /_search
{
  "query": {
    "multi_match": {
      "query":  "this is a test",
      "fields": [ "subject^3", "text*" ]
    }
  }
}
```

Fonctionnalités avancées

- Les caractères joker (*) sont utilisables pour les noms de champs
- Notation ^ pour booster un champ : subject^3 triple le score sur subject
- Plusieurs types : best_fields, most_fields, cross_fields, phrase...

💡 Exemple simple

```
{"multi_match": {
  "query": "full text",
  "fields": ["title","body"]
}}
```

prefix · wildcard · regexp

prefix

Recherche sur le terme sans analyse. Cherche le préfixe exact.

JSON / OpenSearch DSL

```
GET /mon-index/_search
{  "query": {
  "prefix": { "postcode": "W1" }
}}
```

wildcard

Caractères joker dans le terme (* et ?).

JSON / OpenSearch DSL

```
GET /mon-index/_search
{  "query": {
  "wildcard": { "postcode": "W?F*HW" }
}}
```

regexp

Expression régulière sur le terme.

JSON / OpenSearch DSL

```
GET /mon-index/_search
{  "query": {
  "regexp": { "postcode": "W[0-9].+" }
}}
```

⚠ Performance : Ces requêtes peuvent être lentes sur de grands index. Activez index_prefixes sur les champs concernés pour accélérer prefix.

match_phrase

Recherche les documents contenant tous les termes dans le même ordre

JSON / OpenSearch DSL

```
# Phrase exacte
GET /mon-index/_search
{  "query": {
  "match_phrase": { "title": "quick brown fox" }
}}

# Avec flexibilité (slop)
GET /mon-index/_search
{  "query": { "match_phrase": {
  "title": {
    "query": "quick fox",
    "slop":  20
  }
}}}
```

match_phrase

- Analyse la chaîne pour produire une liste de termes
- Ne garde que les documents contenant tous les termes
- Les termes doivent être dans la même position relative

Paramètre slop

- Introduit de la flexibilité dans le phrase matching
- Indique la distance maximale entre les termes (en mots)
- Les termes les plus proches obtiennent un meilleur score

match_phrase_prefix · fuzzy

JSON / OpenSearch DSL

```
# match_phrase_prefix
# Comme match_phrase, dernier mot = préfixe
{
  "match_phrase_prefix": {
    "brand": {
      "query": "johnnie walker bl",
      "max_expansions": 50
    }
  }
}
```

JSON / OpenSearch DSL

```
# fuzzy — Distance de Levenshtein
GET /mon-index/_search
{  "query": {
  "fuzzy": { "text": "surprize" }
}}
```

match_phrase_prefix

- Se comporte comme match_phrase
- Traite le dernier mot comme un préfixe
- max_expansions : limite le nombre d'expansions du préfixe
- Idéal pour l'autocomplétion

fuzzy – Recherche floue

- Gère les erreurs de frappe (typos)
- Utilise la distance de Levenshtein
- prefix_length : caractères initiaux non modifiés
- max_expansions : limite les termes approchants générés

✏ **Atelier 5.1 — Requêtes DSL**

Agrégations

Buckets – Metrics - Imbrication

Introduction aux agrégations

Pourquoi les agrégations ?

- Extrêmement puissantes pour le reporting et les tableaux de bord
- Visualisation de volumes massifs de données en temps réel
- Le reporting s'adapte dynamiquement aux nouvelles données
- Base de OpenSearch Dashboards (équivalent Kibana)

Metric	Bucket	Pipeline
Calculs statistiques (min, max, avg, sum, stats, percentiles)	Regroupe en "seaux" (terms, range, date_histogram, filter)	Opèrent sur les résultats d'autres agrégations

Structure : `{ "aggs": { "mon_agg": { "terms": { "field": "status" } } } }`

Agrégations Metric

JSON / OpenSearch DSL

```
# avg — Durée moyenne
GET /logs/_search
{
  "size": 0,
  "aggs": {
    "duree_moy":  { "avg":   { "field": "duration" } },
    "duree_min":  { "min":   { "field": "duration" } },
    "duree_max":  { "max":   { "field": "duration" } },
    "duree_stats": { "stats": { "field": "duration" } }



}
```

Agrégations Metric

- **avg** – moyenne d'un champ numérique
- **min / max** – valeur minimale ou maximale
- **sum** – somme d'un champ numérique
- **stats** – avg, min, max, sum, count en une seule requête
- **percentiles** – distribution (p50, p95, p99…)

Tip

- Ajoutez **"size": 0** pour ne retourner que les agrégations.
- Combinables avec une **query** pour restreindre le périmètre de calcul.

Réponse — Agrégations Metric

JSON / OpenSearch Response

```
# Réponse de la requête avg / stats
{
  "hits": { "total": { "value": 1240 } },
  "aggregations": {
    "duree_moy":  { "value": 142.7 },
    "duree_min":  { "value": 12.0 },
    "duree_max":  { "value": 998.3 },
    "duree_stats": {
      "count": 1240,
      "min": 12.0,
      "max": 998.3,
      "avg": 142.7,
      "sum": 176948.0
    }
  }
}
```

Réponse — Agrégations Metric

`hits.total.value` — nombre total de documents analysés

`duree_moy.value` — résultat de avg : la moyenne calculée

`duree_min / max.value` — valeur minimale et maximale du champ

`duree_stats` — objet unique retournant count, min, max, avg et sum

Tip

Le champ `"aggregations"` n'apparaît que si au moins une agrégation est définie dans la requête.
Avec `"size": 0`, le tableau `"hits.hits"` est vide — seul le bloc `"aggregations"` est retourné.

Agrégations Bucket

JSON / OpenSearch DSL

```
# terms — Répartition par statut
GET /logs/_search
{
  "size": 0,
  "aggs": {
    "par_statut": {
      "terms": { "field": "status", "size": 5 }
    },
    "par_jour": {
      "date_histogram": { "field": "date",
"calendar_interval": "day" }
    }
  }
}
```

Agrégations Bucket

- **terms** – regroupe par valeur de champ (top N)
- **range** – tranches numériques (0-100, 100-500…)
- **date_histogram** – regroupement par période (day, month…)
- **filter(s)** – Filtre sur les buckets selon des critères

Tip

- Chaque bucket expose son **doc_count**.
- Imbrication : Bucket dans Bucket, ou Metric dans Bucket.
- Limitation du nombre de buckets via size (défaut : 10).

Réponse — Agrégations Bucket

JSON / OpenSearch Response

```
# Réponse terms + date_histogram
{
  "aggregations": {
    "par_statut": {
      "buckets": [
        { "key": "200", "doc_count": 9820 },
        { "key": "404", "doc_count": 1043 },
        { "key": "500", "doc_count": 317 }
      ]
    },
    "par_jour": {
      "buckets": [
        { "key_as_string": "2024-03-01",
          "key": 1709251200000, "doc_count": 482 },
        { "key_as_string": "2024-03-02", ... }
      ]
    }
  }
}
```

Réponse — Agrégations Bucket

`buckets[ ]` — tableau des groupes (un objet par valeur distincte)

`key` — valeur du champ utilisé pour le regroupement (ex: "200", "404")

`doc_count` — nombre de documents dans ce bucket

`key_as_string` — date formatée (uniquement pour date_histogram)

`key (timestamp)` — epoch millis correspondant à la période du bucket

Tip

Chaque bucket peut contenir des **sous-agrégations** imbriquées (metric ou bucket) — elles apparaissent à l'intérieur de l'objet bucket. Le paramètre `"size"` limite le nombre de buckets retournés (défaut : 10).

Agrégations Pipeline

JSON / OpenSearch DSL

```
# max_bucket — Max parmi les buckets
GET /logs/_search
{
  "size": 0,
  "aggs": {
    "ventes_jour": {
      "date_histogram": { "field": "date",
"calendar_interval": "day" },
      "aggs": { "total": { "sum": { "field":
"amount" } } }
    },
    "pic_ventes": {
      "max_bucket": { "buckets_path":
"ventes_jour>total" }
  }
}
```

Agrégations Pipeline — opérer sur d'autres agrégations

- **max_bucket** – valeur max parmi tous les buckets
- **min_bucket / avg_bucket** – min ou moyenne inter-buckets
- **sum_bucket** – somme des valeurs de tous les buckets
- **derivative** – dérivée (variation entre deux buckets)
- **cumulative_sum** – somme cumulée au fil du temps

Tip

- Les pipelines référencent une agrégation parente via **buckets_path**.
- Elles s'appliquent **APRÈS** le calcul des buckets — pas sur les documents bruts.

Réponse — Agrégations Pipeline

JSON / OpenSearch Response

```
# Réponse max_bucket sur ventes_jour>total
{
  "aggregations": {
    "ventes_jour": {
      "buckets": [
        { "key_as_string": "2024-03-01",
          "doc_count": 320,
          "total": { "value": 45200.0 } },
        { "key_as_string": "2024-03-02",
          "doc_count": 410,
          "total": { "value": 68750.5 } }
      ]
    },
    "pic_ventes": {
      "value": 68750.5,
      "keys": ["2024-03-02"]
    }
  }
}
```

Réponse — Agrégations Pipeline

`ventes_jour.buckets` — buckets parentes contenant l'agrégation source (total)

`total.value` — résultat de l'agrégation metric pour chaque bucket

`pic_ventes.value` — valeur maximale trouvée parmi tous les buckets

`pic_ventes.keys[ ]` — clé(s) du bucket où cette valeur max a été trouvée

Tip

Les agrégations pipeline **ne produisent pas de bucket** — elles retournent uniquement une valeur scalaire (`value`) et les clés associées (`keys`).
Le champ `buckets_path` utilise la notation "parent>enfant" pour cibler l'agrégation source.

Agrégations imbriquées

JSON / OpenSearch DSL

```
# Bucket + Metric imbriqués
GET /logs/_search
{
  "size": 0,
  "aggs": {
    "par_statut": {
      "terms": { "field": "status.keyword" },
      "aggs": {
        "duree_moy": { "avg": { "field": "duration_ms" }
}
      }
    }
  }
}
```

Imbrication : Bucket + Metric

- Chaque bucket peut contenir ses propres agrégations
- Un seul appel API : query + bucket + metric
-

Astuce

- size: 0 → ne retourne que les agrégations
- Combinez query pour restreindre le périmètre
- Nommez vos agrégations de façon explicite

Réponse — Bucket + Metric imbriqués

JSON / OpenSearch Response

```
# Réponse : par_statut (terms) + duree_moy (avg)
{
  "aggregations": {
    "par_statut": {
      "buckets": [
        {
          "key": "200", "doc_count": 9820,
          "duree_moy": {
            "value": 132.4
          }
        },
        {
          "key": "404", "doc_count": 1043,
          "duree_moy": { "value": 287.9 }
        },
        {
          "key": "500", "doc_count": 317,
          "duree_moy": { "value": 542.1 }
        }
      ]
    }
  }
}
```

Réponse — Bucket + Metric imbriqués

status = "200"	status = "404"	status = "500"
doc_count	doc_count	doc_count
9 820 docs	**1 043 docs**	**317 docs**
duree_moy.value	duree_moy.value	duree_moy.value
132.4 ms	**287.9 ms**	**542.1 ms**
↳ avg imbriqué	↳ avg imbriqué	↳ avg imbriqué

Chaque objet bucket contient **ses propres agrégations imbriquées** — ici la metric `duree_moy` est calculée séparément pour chaque valeur de statut.

Tip

La metric `duree_moy.value` apparaît **à l'intérieur de chaque bucket** — pas au niveau racine. Plus le code de statut est élevé, plus la durée moyenne est importante ici.

Agrégations imbriquées — Bucket dans Bucket

JSON / OpenSearch DSL

```
# Bucket dans Bucket : statut → jour
GET /logs/_search
{
  "size": 0,
  "aggs": {
    "par_statut": {
      "terms": {
        "field": "status.keyword", "size": 5
      },
      "aggs": {
        "par_jour": {
          "date_histogram": {
            "field": "@timestamp",
            "calendar_interval": "day"
          },
          "aggs": {
            "duree_moy": {
              "avg": { "field": "duration_ms" }
            }
          }
        }
      }
    }
  }
}
```

Imbrication : Bucket dans Bucket

Niveau 1 — **par_statut** (terms)

Niveau 2 — **par_jour** (date_histogram)

Niveau 3 — **duree_moy** (avg metric)

terms → regroupe les docs par valeur de champ (status)
date_histogram → subdivise chaque groupe par période (jour)
avg → calcule la durée moyenne dans chaque sous-groupe

Tip

L'imbrication est **illimitée en théorie** — chaque niveau "aggs" s'imbrique dans le bucket parent. En pratique, 2-3 niveaux suffisent.

Réponse — Bucket dans Bucket

JSON / OpenSearch Response

```
# par_statut > par_jour > duree_moy
{ "aggregations": {
    "par_statut": {
      "buckets": [
        {
          "key": "200", "doc_count": 9820,
          "par_jour": {
            "buckets": [
              {
                "key_as_string": "2024-03-01",
                "doc_count": 482,
                "duree_moy": { "value": 128.7 }
              },
              { "key_as_string": "2024-03-02",
                "doc_count": 391,
                "duree_moy": { "value": 135.2 } }
            ]
          }
        },
        { "key": "404", "doc_count": 1043, "par_jour":
{ ... } }
      ]
    }
  }
}
```

Réponse — Bucket dans Bucket

`aggregations.par_statut.buckets[]` — liste des buckets de 1er niveau (par statut HTTP)

`.key` — valeur du bucket parent ("200", "404"...)

`.doc_count` — nombre total de documents pour ce statut

`.par_jour.buckets[]` — liste des sous-buckets de 2e niveau (par jour)

`.key_as_string` — date formatée du sous-bucket

`.doc_count` — nombre de docs pour ce statut ET ce jour

`.duree_moy.value` — durée moyenne — calculée dans le sous-bucket

Tip

La réponse reflète exactement **la hiérarchie de la requête** — chaque bucket parent contient un objet `par_jour` lui-même contenant ses propres `buckets[]` avec la metric `duree_moy` imbriquée.

Géolocalisation

Geopoint – Requêtes - Agrégations

Géolocalisation dans OpenSearch

OpenSearch permet de combiner la géolocalisation avec les recherches full-text, structurées et les agrégations.

📍 geo_point

Représente un couple latitude/longitude.
Permet principalement le calcul de distance et le filtrage par zone rectangulaire, polygone ou rayon.

```
String : "40.715, -74.011"
Object : { "lat": 40.72, "lon": -73.99 }
Array  : [ -73.983, 40.719 ]   // [lon, lat]
```

🗺️ geo_shape

Définit une zone via le format GeoJSON standard.
Permet de tester l'intersection entre deux zones géographiques.

```
point, linestring, polygon
multipoint, multilinestring, multipolygon
geometrycollection, envelope, circle
```

Mapping & indexation geo_point

OpenSearch DSL

```
# 1. Définir le mapping explicitement
PUT /attractions
{
  "mappings": {
    "properties": {
      "name":     { "type": "text" },
      "location": { "type": "geo_point" }
  } } }
```

Indexation

```
# 3 formats d'indexation possibles
// Format string
PUT /attractions/_doc/1
{ "name": "Chipotle", "location": "40.715, -
74.011" }

// Format objet
PUT /attractions/_doc/2
{ "name": "Pala Pizza",
  "location": { "lat": 40.722, "lon": -73.989 } }

// Format tableau [lon, lat]
PUT /attractions/_doc/3
{ "name": "Mini Munchies",
  "location": [ -73.983, 40.719 ] }
```

Bonne pratique de mapping

Utiliser index_prefixes pour accélérer les requêtes prefix
Activer doc_values: true (défaut) pour les agrégations geo
Préférer le format objet {lat, lon} pour la lisibilité

⚠ Format Array → [longitude, latitude] (ordre GeoJSON). Formats string/objet → latitude, longitude.
Formats string et objet → latitude, longitude.

Filtres géographiques

4 types de filtres permettent d'inclure ou exclure des documents selon leur position

geo_bounding_box

Points dans un rectangle (top_left + bottom_right)

OpenSearch DSL

```
"filter": {
  "geo_bounding_box": {
    "location": {
  "top_left":     { "lat": 40.8, "lon": -74.0 },
  "bottom_right": { "lat": 40.7, "lon": -73.0 }
  } } }
```

geo_distance

Points dans un rayon autour d'un centre (+ tri/score par distance)

OpenSearch DSL

```
"filter": {
  "geo_distance": {
    "distance": "200km",
    "location": { "lat": 40.72, "lon": -73.99 }
  }
}
```

geo_distance_range

Points dans un anneau (distance dans un intervalle)

OpenSearch DSL

```
"filter": {
  "geo_distance_range": {
    "from": "100km",
    "to":   "300km",
    "location": { "lat": 40.72, "lon": -73.99 }
  } }
```

geo_polygon

Points inclus dans un polygone quelconque

OpenSearch DSL

```
"filter": {
  "geo_polygon": {
    "location": { "points": [
      { "lat": 40.73, "lon": -74.1 },
      { "lat": 40.01, "lon": -71.1 },
      { "lat": 50.56, "lon": -90.58 }
    ] } } }
```

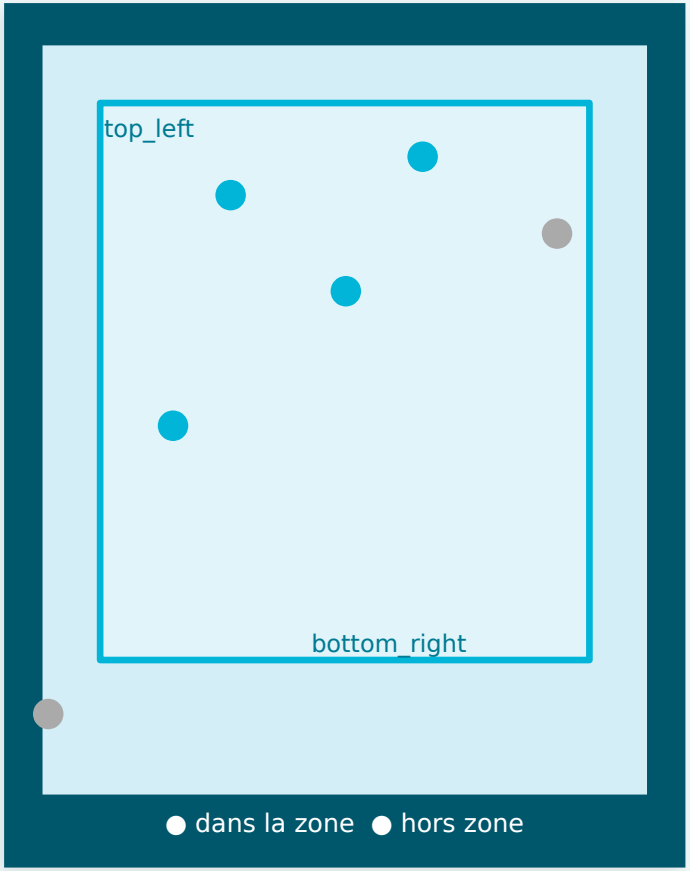
Exemple – geo_bounding_box

OpenSearch DSL

```
GET /attractions/_search
{
  "query": {
    "bool": {
      "filter": {
        "geo_bounding_box": {
          "location": {
            "top_left": {
              "lat": 40.8, "lon": -74.0
            },
            "bottom_right": {
              "lat": 40.7, "lon": -73.0
            }
  } } } } }
```

Contexte filtre (bool.filter)

- Pas de calcul de score → exécution rapide
- Résultats mis en cache par le query cache
- Combinable avec d'autres filtres (term, range…)



Quelques agrégations géographiques

3 types d'agrégations bucket spécifiques aux données géospatiales

geo_distance (bucket)

Groupe les documents dans des ronds concentriques autour d'un point central. Chaque anneau = un bucket.

OpenSearch DSL

```
"aggs": { "per_ring": {
  "geo_distance": {
 "field":  "location",
  "origin": { "lat": 40.712, "lon": -73.988 },
    "unit": "km",
    "ranges": [
      { "from": 0, "to": 1 },
      { "from": 1, "to": 2 }
  ] } } }
```

geohash_grid (bucket)

Groupe les documents par cellule de grille géographique (geohash). Idéal pour les heatmaps sur une carte.

OpenSearch DSL

```
"aggs": { "map_grid": {
  "geohash_grid": {
    "field":     "location",
    "precision": 3 }
} }
```

geo_bounds (métrique)

Retourne un rectangle englobant tous les geo-points. Utile pour définir le niveau de zoom d'une carte.

OpenSearch DSL

```
"aggs": { "viewport": {
  "geo_bounds": {
    "field":      "location",
    "wrap_longitude": true
  }
} }
```

Exemple combiné – Recherche + Filtre + Agrégation géo

OpenSearch DSL

```
GET /attractions/_search
{
  "query": {
    "bool": {
      "must": {
        "match": { "name": "pizza" }   // recherche full-text
      },
      "filter": {                       // filtre géo
        "geo_bounding_box": {
          "location": {
            "top_left":     { "lat": 40.8, "lon": -74.1 },
            "bottom_right": { "lat": 40.4, "lon": -73.7 }
  } } } } },
  "aggs": {                             // agrégation géo
    "per_ring": {
      "geo_distance": {
        "field":  "location", "unit": "km",
        "origin": { "lat": 40.712, "lon": -73.988 },
        "ranges": [{ "from": 0, "to": 1 }, { "from": 1, "to": 2 }]
  } } } }
```

Enchaînement

1 **must : match**

Filtre full-text sur le nom
→ calcule un score

2 **filter : geo_bounding_box**

Restreint à la zone
→ pas de score, mis en cache

3 **aggs : geo_distance**

Groupe les pizzerias
par distance de 1 km

geo_shape – Zones polygonales (GeoJSON)

Permet de requêter des zones complexes et de tester leur intersection

OpenSearch DSL

```
# 1. Mapping explicite requis
PUT /attractions
{
  "mappings": { "properties": {
    "name":     { "type": "text" },
    "location": { "type": "geo_shape" }
} } }

# 2. Indexation d'un polygone GeoJSON
PUT /attractions/_doc/dam_square
{
  "name": "Dam Square, Amsterdam",
  "location": {
    "type": "polygon",
    "coordinates": [[
      [4.892, 52.373], [4.892, 52.372],
      [4.893, 52.372], [4.894, 52.372],
      [4.892, 52.373]  // fermer le polygone
    ] } }
```

Requête geo_shape

```
# 3. Requête d'intersection
GET /attractions/_search
{
  "query": {
    "geo_shape": {
      "location": {
        "shape": {
          "type":   "circle",
          "radius": "1km",
          "coordinates": [4.899, 52.378]
  } } } } }
```

point : Un seul point	**linestring :** Ligne ou courbe
polygon : Zone fermée	**circle :** Rayon autour d'un point
envelope : Rectangle	**geometrycollection :** Combinaison de shapes

✏ Atelier 5.3 — Géo-localisation

Opensearch dashboard

01 Index pattern et Discover

02 DQL et PPL

03 Visualisations et Dashboards

Index Pattern

Filtres – Booléens – Champs imbriqués

Présentation d'OpenSearch Dashboards

OpenSearch Dashboards

- Interface web de visualisation et d'analyse pour OpenSearch
- Fork open-source de Kibana (depuis 2021), maintenu par AWS
- S'exécute sur Node.js

S'adresse également à :

- L'administrateur de la plateforme
- Les développeurs de solutions basées sur OpenSearch
- Les équipes DevOps et observabilité

Visualisation

Graphiques, cartes et tableaux de bord interactifs

Exploration

Discover pour explorer et filtrer vos données

Administration

Gestion des index, alertes et politiques ISM

Tenants

Organisation du contenu

Tenants : espaces de travail isolés pour organiser visualisations, dashboards et index patterns

Tenant Global : partagé avec tous les utilisateurs ayant accès à Dashboards

Tenant Privé : personnel à chaque utilisateur, non visible des autres

Tenants personnalisés : créés par un administrateur, assignables à des rôles (Engineering, Marketing…)

Tags : permettent d'organiser le contenu en complément des tenants

G

Global

Shared with all users

P

Private

Your personal space

E

Engineering

Engineering workspace

S

Security Ops

Security operations

Index Patterns

Rôle des Index Patterns

- Équivalent des Data Views de Kibana
- Indique à Dashboards quels index OpenSearch interroger
- Référence un ou plusieurs index via un wildcard (logs-*, apache-*)
- Définit le champ timestamp pour le filtre temporel (@timestamp par défaut)
- Expose les champs et leurs types (Searchable / Aggregatable)
- Créés dans Stack Management › Index Patterns par un administrateur

Create index pattern

Name

my-index-pattern

Index pattern

logs-*

Time field

@timestamp

Data stream

Index

✓ 3 sources found

Save index pattern to Dashboards

✏ **Atelier 6.1 — Index patterns**

Discover, DQL et PPL

Syntaxe – Agrégations - Alertes

Discover

Explorer ses données avec Discover — histogramme temporel, liste de documents, statistiques sur les champs. Recherches sauvegardables.

Discover

logs-* ▾

🔍 Filter your data using DQL syntax

Last 15 min ▾

Available fields

t @timestamp

t http.status

t url.path

t bytes

t client.ip

t geo.country

Documents (1,295,287) **Field statistics**

```
Jan 3 @ 06:24:27  GET /formation/web/presentation.php  200  16655
Jan 3 @ 06:24:05  GET /Page4047aspxerrorpath=/form...   200  8912
Jan 3 @ 06:24:00  GET /Home/Page404.php                200  8912
```

Recherche dans Discover

Méthodes de recherche

Filtre temporel

Quick select, plages relatives ou absolues, rafraîchissement automatique

Filtres sur données structurées

Filtres visuels sur les champs. Convertibles en Query DSL.

DQL – Dashboards Query Language

Syntaxe native pour recherches textuelles et structurées

PPL / SQL

Piped Processing Language et SQL pour requêtes analytiques

Exemples DQL

Recherche simple

```
http.status_code: 404
```

Plage de valeurs

```
bytes > 10000 AND bytes < 50000
```

Wildcard

```
url.path: "/api/*"
```

Combinaison de critères

```
http.status_code: (400 OR 500)
AND NOT http.method: GET
```

Les recherches peuvent être sauvegardées

DQL – Dashboards Query Language

Rôle de DQL

DQL (Dashboards Query Language) est l'équivalent d'OpenSearch du KQL de Kibana — les syntaxes sont quasi identiques.

Sert uniquement à filtrer les données. DQL n'effectue pas de tri ni d'agrégations.

Full-text + recherche par champ : combine les deux modes en une syntaxe unifiée.

Résultats full-text triables par pertinence dans Discover.

Alternatif : syntaxe Lucene disponible en basculant le filtre language dans Discover.

DQL vs Lucene

Fonctionnalité	DQL	Lucene
Full-text	✓	✓
Recherche par champ	✓	✓
Wildcards	✓	✓
Intervalles	✓	✓
Objets imbriqués	✓	✗
Champs scriptés	✓	✗
Expressions régulières	✗	✓
Fuzzy search	✗	✓

DQL – Exemples de filtrage

Existence d'un champ

Le champ existe

```
http.request.method: *
```

Recherche par valeur

Valeur exacte / full-text

```
http.request.method: GET
```

Plusieurs valeurs (OU implicite)

```
http.request.method: GET POST
```

Sans champ (tous les champs)

```
GET
```

DQL – Exemples de filtrage (2)

Recherche textuelle

Champ contient les mots (ordre libre)

```
http.request.body.content: null pointer
```

Champ contient la phrase exacte

```
http.request.body.content: "null pointer"
```

Phrase dans le body de réponse

```
http.response.body.content.text: "quick brown fox"
```

DQL – Intervalles et wildcards

Intervalles

Principalement utilisés sur les chiffres et les dates :

Inférieur strict

```
http.response.bytes < 10000
```

Intervalle (and)

```
http.response.bytes > 10000
and http.response.bytes <= 20000
```

Date relative

```
@timestamp < now-2w
```

Wildcards *

Le wildcard * filtre par préfixe de valeur ou permet de cibler plusieurs champs :

Préfixe de valeur

```
http.response.status_code: 4*
```

Tous les codes 2xx

```
http.response.status_code: 2*
```

Plusieurs champs

```
datastream.*: logs
```

Champ quelconque

```
*.status_code: 200
```

DQL – Opérateurs booléens

NOT

Négation — exclut les documents correspondant à la condition.

```
NOT http.request.method: GET
```

```
NOT http.response.status_code: 200
```

AND

Les deux conditions doivent être vraies simultanément.

```
http.request.method: GET AND http.response.status_code: 400
```

OR

Au moins une des conditions doit être vraie.

```
http.request.method: GET OR http.response.status_code: 400
```

```
(http.request.method: GET AND http.response.status_code: 200) OR (http.request.method: POST AND
http.response.status_code: 400)
```

DQL – Champs imbriqués (Nested objects)

Requêtes sur objets imbriqués

DQL permet de cibler des champs au sein d'objets ou tableaux imbriqués avec une syntaxe dédiée. La cohérence entre les champs d'un même objet est garantie.

```
{ "user": [
   { "first": "John", "last": "Smith" },
   { "first": "Alice", "last": "White" }
  ]}
```

Syntaxe : champ:{ cond1 and cond2 }

Les accolades garantissent que les deux conditions portent sur le même objet du tableau.

Exemples

Rechercher Alice White (1 niveau)

```
user:{ first: "Alice" and last: "White" }
```

Sans les accolades, DQL pourrait matcher
John White + Alice Smith comme deux résultats distincts.

Chemin complet si 2 niveaux d'imbrication

```
user.names:{ first: "Alice" and last: "White"
}
```

Avec la structure user > names > {first, last},
il faut donner le chemin complet user.names.

⚠ Lucene : fonctions avancées (regex, fuzzy) ne supporte pas les objets imbriqués ni les champs scriptés.

PPL – Piped Processing Language

Principe

PPL est le langage de requêtes analytiques natif d'OpenSearch (équivalent de l'ES|QL d'Elasticsearch).

Commandes enchaînées avec le symbole | (pipe) : chaque commande reçoit le résultat de la précédente.

Une seule requête peut combiner recherche, filtrage, agrégation, calcul et tri.

Disponible dans Discover et dans les Dev Tools via l'API _ppl.

Assistance intégrée : auto-complétion, éditeur multiligne, historique des requêtes.

OpenSearch supporte aussi SQL pour les utilisateurs préférant cette syntaxe.

PPL (OpenSearch) vs ES|QL (Elasticsearch)

Fonctionnalité	PPL	ES|QL
Syntaxe pipe |	✓	✓
Filtre WHERE	✓	WHERE
Agrégation STATS	stats	STATS
Tri	sort	SORT
Sélection colonnes	fields	KEEP
Enrichissement	✓	✓
Visualisation directe	✓	✓
Alertes	✓	✓

PPL – Syntaxe de base

Commande source obligatoire

Lire depuis un index	`search source=opensearch_sample_data_logs`
Alias court (même résultat)	`source=opensearch_sample_data_logs`

Filtrage et sélection

Limiter le nombre de résultats	`source=logs | head 10`
Filtrer sur une condition	`source=logs | where bytes > 10000`
Sélectionner des colonnes	`source=logs | fields @timestamp, bytes, geo.dest`

Tri

Tri descendant	`source=logs | fields bytes, geo.dest | sort - bytes`
Tri ascendant	`source=logs | sort + @timestamp`

PPL – Agrégations avec stats

La commande stats

Permet des calculs statistiques. Combinée avec by pour regrouper les résultats :

```
count()
sum(champ)
avg(champ)
max(champ)
min(champ)
distinct_count(champ)
```

Résultat : tableau de résultats agrégés, directement visualisable dans Discover.

La visualisation Discover s'adapte à la query PPL. Elle est éditable (couleurs, axes) et sauvegardable dans un dashboard.

Exemple complet

```
source=opensearch_sample_data_logs
| stats total_bytes = sum(bytes) by geo.dest
| sort - total_bytes
| head 3
```

Résultat :

total_bytes	geo.dest
2,996,101	CN
2,428,634	IN
1,125,692	US

PPL – Enrichissement (lookup)

La commande lookup

lookup enrichit le jeu de données courant avec des champs provenant d'un autre index (table de référence).

Prérequis : l'index de lookup doit exister dans OpenSearch avec les champs de jointure et les champs à ajouter.

Syntaxe : lookup <index> <champ_jointure> AS <alias> APPEND <champs_à_ajouter>

Alternative ENRICH : OpenSearch supporte également une ENRICH POLICY similaire à Elasticsearch, configurable dans Stack Management.

Exemples PPL

Lookup direct

```
source=opensearch_sample_data_logs
| stats total_bytes=sum(bytes) by geo.dest
| sort - total_bytes | head 3
| lookup countries geo.dest
  APPEND country_name
```

Sans lookup préconfiguré (jointure inline)

```
source=opensearch_sample_data_logs
| JOIN ON geo.dest = country_name
  country_index
| FIELDS geo.dest, country_name,
  country_code
```

ENRICH POLICY (alternative) :

```
source=logs | stats ... | enrich countries
```

Policy créée dans Stack Management › Enrich Policies

PPL – Alertes

Créer une alerte PPL

1. **Depuis Discover**
   Discover › Alerts › Create search threshold rule
2. **Écrire la requête**
   La requête PPL est saisie dans l'éditeur et associée à un champ temporel.
3. **Fenêtre temporelle**
   Définir la fenêtre d'évaluation (ex : toutes les 5 minutes).
4. **Condition**
   Si la requête retourne des documents, l'alerte est déclenchée.
5. **Actions**
   Notification par email, Slack, webhook ou PagerDuty.

Create rule

Name

High traffic

Tag (optional)

Requête PPL

```
source=opensearch_sample_data_logs
| stats total_bytes=sum(bytes) by geo.dest
| sort - total_bytes
| where total_bytes > 10000000
```

Fenêtre : 5 minutes

Query matched ✓

total_bytes	geo.dest
14,888,980	CN
12,974,807	IN

Visualisations et dashboards

Index pattern – Discover – Visualisations - Dashboards

Création de visualisations

1

Accéder à Visualize

Menu Visualize › Create new visualization. Choisir le type de visualisation.

2

Sélectionner la source

Choisir l'Index Pattern contenant les données à visualiser.

3

Configurer métriques & buckets

Métriques (Y-axis : Count, Sum, Avg…) et agrégations de regroupement (X-axis, Split series, Split chart).

4

Ajuster et personnaliser

Paramétrer l'affichage : couleurs, étiquettes, axes, légende, seuils visuels.

5

Sauvegarder

Donner un nom à la visualisation et l'intégrer dans un tableau de bord.

Visualisations disponibles

Bar chart

Histogrammes verticaux / horizontaux, empilés.

Line / Area

Séries temporelles, tendances, évolutions.

Pie / Donut

Répartition proportionnelle entre catégories.

Data Table

Résultats tabulaires multi-niveaux d'agrégation.

Region / Tile Map

Visualisation géographique des données.

Gauge / Metric

Valeur unique mise en avant, jauge de progression.

Tag Cloud

Nuage de mots pondérés par fréquence.

Heat Map

Densité de valeurs sur une grille temporelle.

TSVB

Time Series Visual Builder : métriques avancées.

Vega / Vega-Lite

Visualisations personnalisées en JSON.

Saved search

Recherche Discover intégrée dans un dashboard.

Markdown

Texte enrichi pour documenter un dashboard.

Tableaux de bord

Créer un tableau de bord

Accéder à Dashboard

Menu Dashboard › Create new dashboard

Ajouter des panneaux

Bouton Add — visualisations, saved searches ou Markdown

Disposition

Déplacer et redimensionner par drag & drop

Filtre global

Filtres et sélecteur de temps appliqués à tous les panneaux

Drill-down inter-panneaux

Un clic sur une valeur filtre automatiquement tout le dashboard

Filtres et paramètres interactifs

Filtre temporel

- Présent sur toutes les pages Dashboards
- Quick select : dernières 15 min, 24h, 7 jours…
- Plage relative ou absolue personnalisable
- Rafraîchissement automatique configurable

Filtres sur champs

- Clic sur une valeur → filtre automatique
- Éditables visuellement (is / is not / exists / between)
- Convertibles en Query DSL
- Combinables avec AND sur tous les panneaux

Drill-down / Controls

- Clic dans un graphique → filtre tout le dashboard
- Panneau Controls : inputs interactifs (dropdown, range slider)
- Filtres désactivables sans les supprimer
- Pin filter : filtre persistant entre dashboards

Sauvegarde et partage des dashboards

Sauvegarder

- Bouton Save dans la barre de menu
- Titre et description obligatoires
- Sauvegardé dans le tenant courant
- Versioning via les Saved Objects

Partager

- Share › Get links : lien permanent
- Embed code : iframe intégrable
- Export PNG / PDF (rapports planifiables)
- Short URL pour liens avec filtres complexes

Import / Export

- Stack Management › Saved Objects
- Export JSON : dashboards, visualisations, index patterns
- Import JSON sur un autre cluster
- Promotion entre environnements (dev → prod)

Rapports et alertes

- Rapports PDF/CSV planifiables depuis le dashboard
- Alerting : déclenchement sur seuils OpenSearch
- Notifications : email, Slack, webhook
- Surveillance continue sans ouvrir l'interface

Administration et sécurité

01 Rôles et types de noeuds

02 Sauvegardes et snapshots

03 Sécurité

Rôles et types de nœuds

Types de nœud – HA

Gestion des nœuds – Rôles et types

Dans un cluster OpenSearch, chaque nœud peut avoir un ou plusieurs rôles. Une bonne répartition des rôles est clé pour la stabilité et la performance.

cluster_manager (master)

- Gère l'état du cluster (état des shards, mappings)
- Élit le cluster manager parmi les éligibles
- Ne devrait pas traiter de requêtes utilisateur
- Recommandé : 3 nœuds dédiés (quorum)

data

- Stocke les shards et les données
- Exécute les requêtes CRUD, search, agrégations
- Nœud le plus gourmand en disque et RAM
- Peut être spécialisé : data_hot, data_warm, data_cold

ingest

- Pré-traite les documents avant indexation
- Applique des pipelines (grok, gsub, enrich…)
- Peut être séparé des nœuds data
- Remplacé par des pipelines d'ingestion dédiés

coordinating (client)

- Reçoit les requêtes, les distribue et agrège
- Aucun rôle de stockage ou de management
- Utile pour décharger les nœuds data/master
- Configuré avec une liste vide de rôles

Gestion des nœuds – Configuration opensearch.yml

Configuration par rôle

```
# Nœud cluster_manager dédié
node.name: master-node-1
node.roles: [ cluster_manager ]
cluster.name: mon-cluster
network.host: 10.0.1.10

# Nœud data dédié
node.name: data-node-1
node.roles: [ data ]
path.data: /data/opensearch

# Nœud data chaud (ILM tiering)
node.name: data-hot-1
node.roles: [ data_hot ]
node.attr.temperature: hot

# Nœud coordinating uniquement
node.name: coord-node-1
node.roles: []
```

API de gestion du cluster

Santé du cluster

```
GET /_cluster/health?pretty
```

Liste des nœuds

```
GET /_cat/nodes?v
```

Infos détaillées nœuds

```
GET /_nodes/stats
```

État des shards

```
GET /_cat/shards?v
```

Paramètres cluster

```
GET /_cluster/settings
```

Rerouter un shard

```
POST /_cluster/reroute
{"commands":[{"move":{"index":"logs",
  "shard":0,"from_node":"n1","to_node":"n2"}}]}
```

Haute disponibilité – Réplication et allocation

La HA dans OpenSearch repose sur la réplication des shards et les politiques d'allocation pour garantir qu'aucune donnée n'est perdue en cas de défaillance.

Shards primaires & réplicas

Principe

- Un index est découpé en N shards primaires
- Chaque primaire a R réplicas (copies)
- Les réplicas sont placés sur des nœuds différents
- Lecture possible sur primaires ET réplicas
- Écriture toujours sur le primaire, répliquée ensuite

```
# Définir les réplicas à la création
PUT /mon-index
{
  "settings": {
    "number_of_shards": 3,
    "number_of_replicas": 1
  }
}
# Modifier sur un index existant
PUT /mon-index/_settings
{
  "number_of_replicas": 2
}
```

Allocation filtering

Règles d'allocation

- Contrôle sur quels nœuds les shards sont placés
- include: au moins un tag doit correspondre
- require: tous les tags doivent correspondre
- exclude: aucun shard sur ces nœuds

```
# Attribut nœud dans opensearch.yml
node.attr.zone: zone-a
# Allocation sur les nœuds de zone-a uniquement
PUT /mon-index/_settings
{
  "index.routing.allocation.require.zone": "zone-a"
}
# Exclure un nœud de l'allocation (décommission)
PUT /_cluster/settings
{
  "transient": {
    "cluster.routing.allocation.exclude._name": "data-
node-3"
  }
}
```

Haute disponibilité – Rack / Zone Awareness

Le Rack Awareness garantit que les copies d'un shard (primaire et réplicas) sont distribuées sur des zones physiques distinctes, évitant toute perte de données en cas de défaillance d'une zone.

Configuration

```
# Sur chaque nœud — opensearch.yml
node.attr.zone: zone-a    # (ou zone-b, zone-c)
cluster.name: prod-cluster

# Sur les nœuds cluster_manager
cluster.routing.allocation.awareness.attributes: zone

# Forcer la répartition équilibrée (forced awareness)
cluster.routing.allocation.awareness.force.zone.values:
zone-a,zone-b,zone-c
```

⚠ Sans forced awareness, si une zone disparaît, OpenSearch peut créer plusieurs réplicas sur les zones restantes et surcharger ces nœuds.

Répartition des shards

Zone A	Zone B
data-1 P0 P1	data-3 R1 R2
data-2 P2 R0	data-4 P0r P1r

P0 = Shard primaire 0 **R0** = Réplica du shard 0

Sauvegarde et snapshot

Repositories – API - Automatisation

Sauvegardes et snapshots – Repositories

Les snapshots OpenSearch sont des sauvegardes incrémentales des index. Ils sont stockés dans un repository (dépôt) configuré au préalable.

fs (filesystem)

- Chemin NFS/local partagé entre tous les nœuds
- path.repo doit être configuré dans opensearch.yml
- Simple à mettre en place, adapté aux labs/petits clusters

s3 (Amazon S3 / MinIO)

- Plugin opensearch-repository-s3 requis
- Compatible S3 : MinIO, Ceph RGW, OVH Object Storage…
- Solution de référence pour les clusters en production

gcs (Google Cloud Storage)

- Plugin opensearch-repository-gcs requis
- Stockage objet managé sur GCP
- Idéal si cluster sur GKE/GCP

azure (Azure Blob Storage)

- Plugin opensearch-repository-azure requis
- Stockage objet managé sur Azure
- Idéal si cluster sur AKS/Azure

Sauvegardes et snapshots – Création et restauration

Enregistrer un repository & créer un snapshot

```
# 1. Enregistrer un repository S3
PUT /_snapshot/backup-s3
{
  "type": "s3",
  "settings": {
    "bucket": "opensearch-backups",
    "region": "eu-west-1",
    "base_path": "prod-cluster"
  }
}

# 2. Créer un snapshot manuel
PUT /_snapshot/backup-s3/snapshot-2024-01-15
{
  "indices": "logs-*,metrics-*",
  "ignore_unavailable": true,
  "include_global_state": false
}

# 3. Suivre la progression
GET /_snapshot/backup-s3/snapshot-2024-01-15/_status
```

Restaurer un snapshot

```
# Lister les snapshots disponibles
GET /_snapshot/backup-s3/_all

# Restaurer un index spécifique
POST /_snapshot/backup-s3/snapshot-2024-01-15/_restore
{
  "indices": "logs-2024.01.14",
  "rename_pattern":  "(.+)",
  "rename_replacement": "restored-$1",
  "index_settings": {
    "index.number_of_replicas": 0
  }
}

# Surveiller la restauration
GET /_cat/recovery?v&active_only=true
```

⚠ Avant restauration :
- Fermer ou supprimer l'index cible s'il existe déjà
- Les réplicas peuvent être remis à 0 pour accélérer la restauration, puis réactivés ensuite

Sauvegardes – Snapshot Management (automatisation)

Le plugin Index Management (ISM) intègre une fonctionnalité de Snapshot Management pour automatiser la création et la rétention des snapshots.

Via OpenSearch Dashboards

1. Index Management → Snapshot Management → Create policy
2. Définir le repository cible (backup-s3)
3. Planifier via une expression cron (ex: 0 1 * * * = chaque nuit à 1h)
4. Configurer la rétention (nb max, durée max)
5. Activer et surveiller dans Dashboards

Via API

```
POST /_plugins/_sm/policies/daily-backup
{
  "description": "Sauvegarde quotidienne",
  "creation": {
    "schedule": {
      "cron": {
        "expression": "0 1 * * *",
        "timezone": "Europe/Paris"
      }
    },
    "time_limit": "1h"
  },
  "deletion": {
    "schedule": {
      "cron": {
        "expression": "0 2 * * *",
        "timezone": "Europe/Paris"
      }
    },
    "condition": {
      "max_count": 30,
      "max_age": "30d",
      "min_count": 5
    }
```

Sécurité

TLS – RBAC – Documents – Intégration Realms - Audit

Sécurité – Certificats TLS

OpenSearch utilise deux couches TLS indépendantes qu'il faut configurer séparément dans opensearch.yml.

Transport layer (inter-nœuds)

Chiffre les communications entre nœuds du cluster. Obligatoire.

```
plugins.security.ssl.transport.pemcert_filepath:
node.pem
plugins.security.ssl.transport.pemkey_filepath: node-
key.pem
plugins.security.ssl.transport.pemtrustedcas_filepath:
root-ca.pem
plugins.security.ssl.transport.enforce_hostname_verific
ation: false
```

REST layer (API HTTP)

Chiffre les communications client ↔ cluster (HTTPS). Recommandé.

```
plugins.security.ssl.http.enabled: true
plugins.security.ssl.http.pemcert_filepath: http.pem
plugins.security.ssl.http.pemkey_filepath: http-key.pem
plugins.security.ssl.http.pemtrustedcas_filepath: root-
ca.pem
```

Appliquer la configuration de sécurité

```
# Après modification des certificats ou de la configuration security
./plugins/opensearch-security/tools/securityadmin.sh \
  -cd config/opensearch-security/ \
  -icl -nhnv \
  -cacert config/root-ca.pem \
  -cert config/admin.pem \
  -key config/admin-key.pem
```

Sécurité – Génération des certificats TLS

Créer une CA auto-signée et signer les certificats nœuds et admin avec OpenSSL

Commandes OpenSSL

```
# 1. CA root
openssl genrsa -out root-ca-key.pem 2048
openssl req -new -x509 -sha256 -key root-ca-key.pem \
  -subj '/CN=root-ca' -out root-ca.pem -days 730

# 2. Certificat admin  (pour securityadmin.sh)
openssl genrsa -out admin-key-temp.pem 2048
openssl pkcs8 -inform PEM -outform PEM -in admin-key-temp.pem \
  -out admin-key.pem -nocrypt
openssl req -new -key admin-key.pem -subj '/CN=admin' -out admin.csr
openssl x509 -req -in admin.csr -CA root-ca.pem -CAkey root-ca-key.pem \
  -CAcreateserial -sha256 -out admin.pem -days 730

# 3. Certificat nœud  (SAN = noms de tous les conteneurs)
openssl genrsa -out node-key-temp.pem 2048
openssl pkcs8 -inform PEM -outform PEM -in node-key-temp.pem \
  -out node-key.pem -nocrypt
```

Rôle de chaque certificat

root-ca.pem
CA racine auto-signée. Signe tous les autres certificats. Doit être distribuée sur chaque nœud comme CA de confiance (pemtrustedcas_filepath).

admin.pem + key
Certificat super-admin utilisé uniquement par securityadmin.sh pour pousser la configuration de sécurité. Ne jamais mettre sur les nœuds.

node.pem + key
Certificat de nœud utilisé pour les deux couches TLS (transport inter-nœuds ET REST HTTP). Doit inclure les SANs de tous les conteneurs.

💡 **Tip** En production, utiliser une PKI interne ou cfssl / opensearch-onboarding pour gérer le cycle de vie des certificats. Le même certificat node peut être partagé entre tous les nœuds en lab, mais en production chaque nœud doit avoir le sien.

Sécurité – RBAC : Rôles et utilisateurs

Le modèle RBAC (Role-Based Access Control) d'OpenSearch Security permet de contrôler finement les accès aux index, aux champs et aux données (Document Level Security).

Créer un rôle et un utilisateur

```
# Créer un rôle avec permissions sur un index
PUT /_plugins/_security/api/roles/logs-reader
{
  "cluster_permissions": ["cluster_composite_ops_ro"],
  "index_permissions": [{
    "index_patterns": ["logs-*"],
    "allowed_actions":
["read","indices:data/read/search"]
  }]
}

# Créer un utilisateur interne
PUT /_plugins/_security/api/internalusers/alice
{
  "password": "SecretP@ss1",
  "backend_roles": [],
  "attributes": { "dept": "ops" }
}

# Mapper l'utilisateur au rôle
PUT /_plugins/_security/api/rolesmapping/logs-reader
{ "users": ["alice"] }
```

Rôles prédéfinis

`all_access`
Accès total au cluster (super-admin)

`security_manager`
Gestion des utilisateurs et rôles

`kibana_user`
Accès à OpenSearch Dashboards

`readall`
Lecture seule sur tous les index

`logstash`
Écriture pour les agents d'ingestion

`snapshot_restore`
Création et restauration de snapshots

💡 Document Level Security (DLS) : filtrer les documents visibles par rôle via une query DSL dans les index permissions.

Document & Field Level Security (DLS / FLS)

Restreindre les documents visibles et les champs accessibles par rôle, sans dupliquer les index.

DLS — Document Level Security

Filtrer les documents via une query DSL dans les index_permissions du rôle.

```
# Seuls les logs de niveau 'error' sont visibles
PUT /_plugins/_security/api/roles/logs-errors
{
  "index_permissions": [{
    "index_patterns": ["logs-*"],
    "allowed_actions": ["read"],
    # La query DLS est injectée côté serveur
    "dls": "{\"term\":{\"level\":\"error\"}}"
  }]
}
```

FLS — Field Level Security

Inclure (~) ou exclure les champs retournés dans chaque document.

```
# Masquer les champs sensibles (IP, email…)
PUT /_plugins/_security/api/roles/logs-masked
{
  "index_permissions": [{
    "index_patterns": ["logs-*"],
    "allowed_actions": ["read"],
    "fls": [
      "~client_ip", # exclure ce champ
      "~user.email"  # exclure ce champ
    ] }] }
```

DLS — granularité document

- Filtre sur les documents retournés (WHERE)
- Query DSL standard injectée côté serveur
- L'utilisateur ne peut pas contourner le filtre
- Exemple : chaque équipe ne voit que ses propres logs

FLS — granularité champ

- Filtre sur les colonnes retournées (SELECT)
- Inclusion (liste des champs autorisés) ou exclusion (~champ)
- Compatible avec DLS : les deux peuvent se combiner
- Exemple : masquer les IPs et emails dans les logs

Sécurité – Backends d'authentification

OpenSearch Security supporte plusieurs backends d'authentification (authc) configurables dans config.yml. Ils sont évalués dans l'ordre de priorité défini.

Basic (utilisateurs internes)

- Stockage des users dans le Security Index interne
- Hash bcrypt des mots de passe
- Idéal pour les comptes de service et les labs

LDAP / Active Directory

- Connexion à un annuaire LDAP/AD existant
- Synchronisation automatique des groupes → rôles
- Authentification et autorisation LDAP distinctes

SAML 2.0 (SSO)

- SSO via un Identity Provider (Okta, Keycloak, ADFS…)
- Utilisé principalement pour OpenSearch Dashboards
- Flux de redirection standard SAML POST/Redirect

OIDC / JWT

- OpenID Connect ou tokens JWT signés
- Compatible Keycloak, Auth0, Azure AD, Google…
- Validation via JWKS endpoint ou secret partagé

```
# config/opensearch-security/config.yml — activer plusieurs backends
authc:
  ldap_auth: { order: 1, http_authenticator: {type: basic}, authentication_backend: {type: ldap, config: {…}}}
  internal_auth: { order: 2, http_authenticator: {type: basic}, authentication_backend: {type: intern}}
```

Sécurité – Audit Logging

Tracer qui a accédé à quoi et quand — essentiel pour la conformité (RGPD, SOC2, ISO 27001).

Configuration — opensearch-security/audit.yml

```yaml
# Destination : index OpenSearch interne
config:
  enabled: true
  audit:
    type: "internal_opensearch"
    log_request_body: false
    resolve_indices: true

# Compliance — traçabilité lecture/écriture
compliance:
  enabled: true
  write_log_diffs: true
true
  read_watched_fields:
    "logs-*": ["client_ip", "user.email"]

# Consulter les logs d'audit
GET /.opensearch-security-auditlog*/_search
```

Catégories d'événements tracées

FAILED_LOGIN
Tentatives d'authentification échouées — détection de brute-force

AUTHENTICATED
Connexions réussies — qui s'est connecté, quand, depuis quelle IP

GRANTED_PRIVILEGES
Actions autorisées sur les index (GET, PUT, DELETE…)

MISSING_PRIVILEGES
Tentatives d'accès refusées — qui essaie d'accéder à quoi

INDEX_EVENT
Création, suppression, modification de mapping d'index

💡 **Stockage** Par défaut les logs sont écrits dans un index .opensearch-security-auditlog-YYYY.MM.dd. Ils peuvent aussi être redirigés vers log4j (fichier) ou un webhook externe. Appliquer une politique ISM sur ces index pour gérer leur rétention.

Administration via OpenSearch Dashboards

OpenSearch Dashboards embarque plusieurs modules d'administration accessibles depuis le menu Stack Management et le menu principal.

Index Management (ISM)

- Politiques de cycle de vie : hot → warm → cold → delete
- Rollover, force_merge, shrink, snapshot par état
- ISM Templates : applique une policy aux nouveaux index
- Suivi des managed indexes et de leur état courant

Snapshot Management

- Création et planification (cron) des politiques de snapshot
- Visualisation par repository et navigation dans les snapshots
- Restauration guidée depuis l'interface
- Rétention configurable : max_count, max_age, min_count

Security Plugin

- Gestion des utilisateurs internes (create / edit / delete)
- Création et assignation des rôles (RBAC)
- Mapping rôles ↔ groupes backend (LDAP, SAML…)
- Audit logs : qui a accédé à quoi, quand

Dev Tools & Alerting

- Console interactive : Query DSL, PPL, SQL
- Alerting > Monitors : seuils, anomalies ML
- Notifications : email, Slack, webhook, PagerDuty
- Observability : traces et métriques OpenTelemetry

ISM – Index State Management

Concepts clés

- **Policy** — ensemble d'états et de transitions définissant le cycle de vie d'un index
- **State** — étape du cycle : hot, warm, cold, delete (noms libres)
- **Action** — opération exécutée en entrant dans un état (rollover, force_merge, allocation, delete…)
- **Transition** — condition de passage vers l'état suivant (min_index_age, min_size, min_doc_count)
- **ISM Template** — applique automatiquement une policy aux nouveaux index matchant un pattern
- **Managed Index** — index piloté par ISM, visible dans Dashboards > Index Management

Astuce ISM est l'équivalent OpenSearch des **ILM Policies** d'Elasticsearch. Les deux concepts sont quasi identiques.

PUT /_plugins/_ism/policies/logs-lifecycle

```
# Structure minimale d'une policy ISM
{

"policy": {

"description": "Cycle de vie logs"
    "default_state": "hot"
    "states": [
      {

"name": "hot", "actions": [{

"rollover": {

"min_size": "50gb", "min_doc_count": 5000000
          }
        }],

"transitions": [{

"state_name": "warm",

"conditions": { "min_index_age": "7d }
        }]
      },
      { ... état warm ... },
      { ... état cold ... },
      { ... état delete ... }
    ],

"ism_template": [{
```

ISM – Politique hot → warm → cold → delete

HOT
Écriture active
rollover ≥50GB/7j

▶

WARM
Lecture seule
après 30 j

▶

COLD
Accès rare
après 60 j

▶

DELETE
Suppression
après 90 j

État warm — actions

```
"actions": [{
"force_merge": { "max_num_segments": 1 } },
  { "index_settings": {
"index.number_of_replicas": 1,
"index.blocks.write": true
  } },
  { "allocation": {
"require": { "temperature": "warm" } } }]
```

État cold + delete

```
# État cold
"actions": [{ "allocation": { "require": { "temperature":
"cold" } },

"index_settings": {"index.number_of_replicas": 0 } }],
# État delete (snap avant)
"actions": [{ "snapshot": { "repository": "backup-s3" } },
  {
"delete": {} }],
"transitions": []  // état final
```

Actions ISM disponibles

rollover Crée un nouvel index quand taille/âge/doc atteint le seuil
force_merge Réduit les segments Lucene (optimise la lecture)
shrink Réduit le nombre de shards primaires
index_settings Modifie les settings de l'index (réplicas, write block…)
allocation Déplace l'index sur des nœuds avec un attribut donné
snapshot Déclenche un snapshot avant suppression
delete Supprime définitivement l'index
close / open Ferme (libère RAM) ou rouvre l'index

Monitoring du cluster via Dashboards & API

Cluster Health

- **`green`** tous les shards primaires ET réplicas assignés
- **`yellow`** primaires OK, certains réplicas non assignés
- **`red`** au moins un shard primaire manquant (perte de données possible)
- `GET /_cluster/health?pretty`
- `GET /_cluster/stats`
- Widget de santé accessible depuis le menu Dashboards

Cat APIs — diagnostics

- `GET /_cat/nodes?v&h=name,role,heap.percent,cpu`
- `GET /_cat/indices?v&s=store.size:desc`
- `GET /_cat/shards?v&h=index,shard,prirep,state,node`
- `GET /_cat/allocation?v`
- `GET /_cat/recovery?v&active_only=true`
- `GET /_cat/pending_tasks?v`

Alerting & Observability

- Performance Analyzer : CPU, latence, débit en temps réel
- Alerting > Monitors : surveille index ou métrique
- Triggers : seuil, anomalie ML, clustering
- Destinations : email, Slack, webhook, PagerDuty
- Observability : traces et métriques (OpenTelemetry)

Opérations courantes d'administration

Maintenance des index

```
# Force merge des segments (après suppressions)
POST /logs-2024.*/_forcemerge?max_num_segments=1

# Fermer / rouvrir un index (libère RAM)
POST /logs-2024.01.*/_close
POST /logs-2024.01.*/_open

# Flush (écriture translog → disque)
POST /logs-*/_flush

# Réindexer vers un nouvel index
POST /_reindex
{ "source": { "index": "logs-old" },

"dest": { "index": "logs-new" } }
```

Circuit breakers & JVM

```
# État des circuit breakers
GET /_nodes/stats/breaker

# Mémoire JVM par nœud
GET /_nodes/stats/jvm

# Quorum split-brain (opensearch.yml)
discovery.zen.minimum_master_nodes: 2
# Formule : (nb_managers / 2) + 1

# Voir tâches en cours
GET /_tasks?detailed=true&actions=*search
GET /_tasks?detailed=true&actions=*reindex
```

Allocation & exclusion de nœud

```
# Exclure un nœud (décommission progressif)
PUT /_cluster/settings
{ "transient": {

"cluster.routing.allocation.exclude._name": "data-node-3"
} }

# Désactiver l'allocation (rolling restart)
PUT /_cluster/settings
{ "transient": {

"cluster.routing.allocation.enable": "primaries"
} }
# Remettre à "all" après redémarrage
```

Procédure : Rolling Restart

1. Désactiver l'allocation : cluster.routing.allocation.enable → primaries
2. Flush : POST /_flush
3. Arrêter le nœud (graceful shutdown)
4. Mettre à jour config / upgrader si besoin
5. Redémarrer et attendre qu'il rejoigne le cluster
6. Vérifier GET /_cat/nodes — nœud present
7. Réactiver l'allocation → all
8. Attendre green avant nœud suivant

Monitoring

01 Supervision et optimisation

02 Performance Analyser

03 Gestion du stockage

Supervision et optimisation

API _cat – Opensearch Dashboard – Alerting – Slow log

Analyse des performances – API _cat et _cluster

L'API _cat offre des vues lisibles sur l'état du cluster. C'est le premier réflexe pour diagnostiquer un problème de performance ou d'allocation.

Commandes _cat essentielles

Santé globale

```
GET /_cluster/health?pretty
```

Nœuds + charge

```
GET /_cat/nodes?v
  &h=name,heap.percent,cpu,load_1m,disk.avail
```

Index + taille

```
GET /_cat/indices?v&s=store.size:desc
```

Shards non assignés

```
GET /_cat/shards?v
  &h=index,shard,prirep,state,node&s=state
```

Tâches en cours

```
GET /_cat/tasks?v&detailed
```

Threads pools

```
GET /_cat/thread_pool?v
  &h=name,active,queue,rejected
```

Diagnostics avancés

```
# Statistiques détaillées par nœud
GET /_nodes/stats/indices,os,jvm,process

# Index les plus lourds
GET /_cat/indices?v&s=pri.store.size:desc
  &h=index,docs.count,store.size,pri.store.size

# Cause des shards non assignés
GET /_cluster/allocation/explain
{
  "index": "mon-index",
  "shard": 0,
  "primary": true
}

# Requêtes en attente
GET /_tasks?actions=*search*&detailed=true

# Utilisation des segments
GET /_cat/segments?
v&h=index,shard,segment,size,docs.count
```

Administration dans OpenSearch Dashboards – Stack Management

Stack Management centralise toutes les fonctions d'administration depuis l'interface web — index, templates, snapshots, alertes, et outils de diagnostic.

Stack Management — sections clés

Index Management

Gérer les index, politiques ISM, templates. Visualiser l'état ISM par index. Déclencher un rollover ou une transition manuellement.

Index Patterns

Créer / modifier les index patterns utilisés par Discover et Visualize. Rafraîchir les champs après un changement de mapping.

Snapshot Management

Créer des repositories (S3, filesystem). Planifier des snapshots automatiques (SLM). Restaurer depuis l'interface.

Alerting

Créer des monitors sur seuils (cluster health, heap, latence). Configurer destinations : email, Slack, webhook PagerDuty.

Dev Tools – outils intégrés

```
# Console — requêtes REST interactives
GET /_cluster/health
GET /_cat/indices?v
POST /mon-index/_search
{ "query": { "match_all": {} } }

# Search Profiler — analyser le temps
# d'exécution d'une requête
# Bouton Profile dans la Console
```

Index Management UI — workflow ISM

```
# Depuis Index Management → Policies
# → Apply policy to index
# Vérifier l'état ISM d'un index
GET /_plugins/_ism/explain/mon-index
# Forcer une transition manuelle
POST /_plugins/_ism/retry/mon-index
{ "state": "warm" }
```

💡 *La Console Dev Tools supporte l'autocomplétion (Ctrl+Espace) et le Search Profiler visualise graphiquement quel composant de la requête prend du temps.*

Alerting – Monitors, triggers et destinations

Le plugin Alerting permet de surveiller en continu l'état du cluster et des données, et de notifier via email, Slack ou webhook dès qu'un seuil est franchi.

Monitor
Requête planifiée

Trigger
Condition de seuil

Action
Notification déclenchée

Destination
Email · Slack Webhook

Monitor — cluster health (via API)

```
POST /_plugins/_alerting/monitors
{
  "name": "Cluster Health Monitor",
  "type": "monitor",
  "enabled": true,
  "schedule": {
    "period": { "interval": 1, "unit": "MINUTES" }
  },
  "inputs": [{
    "search": {
      "indices": [".opensearch-alerting*"],
      "query": {
        "size": 1,
        "query": { "match_all": {} }
      }
    }
```

Cas d'usage recommandés

Cluster status ≠ green
Trigger dès que status=yellow ou red. Priorité critique — notifier immédiatement.

Heap JVM > 80%
Surveiller via _nodes/stats. Anticiper les OOM avant qu'ils ne surviennent.

Shards non assignés > 0
_cluster/health.unassigned_shards. Données inaccessibles si non résolus.

Espace disque < 20%
disk.avail depuis _cat/nodes. Opensearch bloque les écritures à 5% restant.

Latence search P95 > 1s

💡 *Depuis Dashboards → Alerting → Monitors → Create monitor : interface graphique pour créer des monitors sans écrire de JSON — extraction extraction, trigger visuel et test en un clic.*

Analyse des performances – Slow Logs

Les slow logs enregistrent les requêtes et indexations qui dépassent un seuil configurable. Ils sont indispensables pour identifier les requêtes problématiques en production.

Configuration des slow logs

```
# Activer les slow logs sur un index
PUT /mon-index/_settings
{
  "index.search.slowlog.threshold.query.warn":  "5s",
  "index.search.slowlog.threshold.query.info":  "2s",
  "index.search.slowlog.threshold.query.debug":
"500ms",
  "index.search.slowlog.threshold.fetch.warn":  "1s",
  "index.search.slowlog.level": "info",

  "index.indexing.slowlog.threshold.index.warn": "5s",
  "index.indexing.slowlog.threshold.index.info": "2s",
  "index.indexing.slowlog.level": "info",
  "index.indexing.slowlog.source": "1000"
}
```

⚠ En production, commencer avec des seuils élevés (warn: 5s) et les réduire progressivement. Les slow logs peuvent générer un volume important.

Lecture et niveaux de logs

```
# Emplacement des logs
/var/log/opensearch/<cluster>_index_search_slowlog.log
/var/log/opensearch/
<cluster>_index_indexing_slowlog.log

# Exemple de ligne slow log
[2024-01-15T14:32:01][WARN][index.search.slowlog.query]
[logs-2024.01.15][0]
  took[6.2s], took_millis[6234],
  search_type[QUERY_THEN_FETCH],
  source[{"query":{"match":{"msg":"error"}}}]
```

Niveaux de logs OpenSearch

ERROR	Erreur critique nécessitant une intervention
WARN	Anomalie non bloquante, à surveiller
INFO	Événements normaux du cycle de vie
DEBUG	Détails techniques (verbeux)

Performance Analyser

Activation – Analyse – Opensearch dashboard

Performance Analyzer – Activation et métriques

Performance Analyzer est activé par défaut dans OpenSearch. Il expose une API REST sur le port 9600 et s'intègre à Dashboards via le plugin PerfTop.

Activation et requêtes

```
# Vérifier l'état de l'agent
GET http://<node>:9600/
  _plugins/_performanceanalyzer/cluster/config

# Activer / désactiver
POST http://<node>:9600/
  _plugins/_performanceanalyzer/cluster/config
{ "enabled": true }

# Requête de métriques brutes
GET http://<node>:9600/
  _plugins/_performanceanalyzer/metrics
  ?metrics=CPU_Utilization,Heap_Used
  &dim=ShardID,IndexName&nodes=all

# Installer et lancer PerfTop (CLI)
npm install -g @opensearch-project/perftop
```

💡 PerfTop propose des dashboards prédéfinis : ClusterOverview, NodeAnalysis, SearchLatency, MemoryAnalysis.

Métriques disponibles (sélection)

JVM / GC

- Heap_Used, Heap_Max
- GC_Collection_Event, GC_Collection_Time
- Paging_MajfltRate

CPU / OS

- CPU_Utilization
- IO_ReadThroughput, IO_WriteThroughput
- Thread_Blocked_Time

Indexation

- Indexing_ThrottleTime
- Shard_BulkDocs, Indexing_Docs
- Segments_Count, Segments_Memory

Recherche

- Latency (search / fetch)
- Cache_Query_Hit / Miss
- ShardEvents, HTTP_RequestDocs

Performance Analyzer – Root Cause Analysis (RCA)

Le framework RCA analyse en continu les métriques de Performance Analyzer pour détecter automatiquement les causes racines des dégradations de performance.

Pipeline d'analyse RCA

1. **Collecte**
   Métriques brutes via Performance Analyzer (port 9600)
2. **Symptômes**
   Détection : haute latence, GC excessif, CPU saturé…
3. **Analyse**
   Corrélation des métriques → identification de la cause
4. **Résultat**
   Verdict exposé via l'API RCA ou dans Dashboards

API RCA – Analyses disponibles

```
# Lister les analyses RCA disponibles
GET http://<node>:9600/
  _plugins/_performanceanalyzer/rca

# Analyse haute utilisation heap
GET http://<node>:9600/
  _plugins/_performanceanalyzer/rca
  ?rca=HighHeapUsageClusterRca

# Analyses disponibles (sélection)
HighHeapUsageClusterRca   → heap saturé
HotShardClusterRca        → shard surchargé
HotNodeClusterRca         → nœud surchargé
HighCPUClusterRca         → CPU saturé
SlowQueryClusterRca       → requêtes lentes
```

Correspondances symptôme → cause

⚠ **Latence dégradée**	→ Hot shard → rééquilibrer
⚠ **GC fréquent**	→ Heap trop petit → réduire les caches
⚠ **CPU saturé**	→ Trop de shards sur un nœud
⚠ **I/O élevé**	→ Merges excessifs → throttle

Supervision avec OpenSearch Dashboards – Observability

Le plugin Observability centralise logs, métriques et traces dans une interface unifiée, en s'appuyant sur les standards OpenTelemetry (OTel).

Logs Explorer

- Recherche full-text dans les logs via PPL
- Filtres interactifs par champ, niveau, service
- Corrélation avec les traces et métriques
- Sauvegarde de requêtes fréquentes

Trace Analytics

- Visualisation des traces distribuées (OTel)
- Latence par service, opération, span
- Identification des goulots d'étranglement
- Compatible Jaeger et Data Prepper

Métriques (Metrics)

- Ingestion via Data Prepper ou OTel Collector
- Graphiques de séries temporelles
- Corrélation avec les logs associés
- Format Prometheus compatible

Notebooks

- Combine visualisations, requêtes PPL/SQL, markdown
- Idéal pour les rapports de post-mortem
- Partage en équipe, exportable en PDF
- Exécution pas à pas des cellules

Gestion du stockage

Activation – Analyse – Opensearch dashboard

Scaling – Ajout de nœuds et rééquilibrage

OpenSearch supporte le scaling horizontal à chaud : les nouveaux nœuds rejoignent le cluster et les shards sont automatiquement redistribués selon les règles d'allocation.

Ajouter un nœud data

```
# opensearch.yml du nouveau nœud
cluster.name: prod-cluster
node.name: data-node-5
node.roles: [ data ]
network.host: 10.0.1.15
discovery.seed_hosts:
  - 10.0.1.10
  - 10.0.1.11
  - 10.0.1.12

# Démarrer OpenSearch →
# le nœud rejoint le cluster automatiquement

# Surveiller le rééquilibrage
GET /_cat/recovery?v&active_only=true
GET /_cluster/health
```

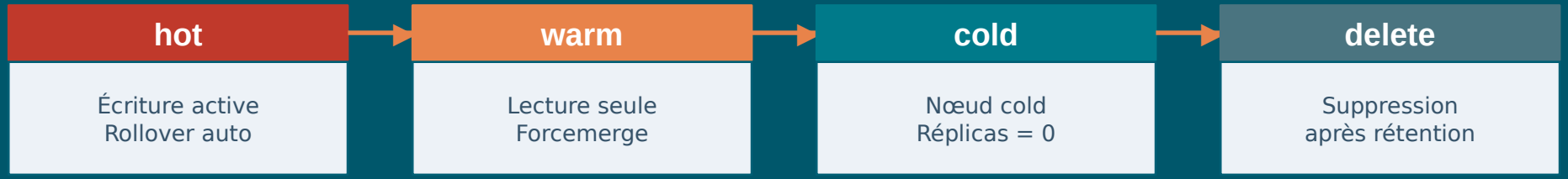
⚠ Le rééquilibrage peut impacter les performances. En prod, limiter la vitesse :
cluster.routing.allocation.node_concurrent_recoveries: 2
indices.recovery.max_bytes_per_sec: 40mb

Contrôle de l'allocation

```
# Désactiver le rééquilibrage pendant maintenance
PUT /_cluster/settings
{
  "transient": {
    "cluster.routing.rebalance.enable": "none"
  }
}
# Réactiver après maintenance
PUT /_cluster/settings
{
  "transient": {
    "cluster.routing.rebalance.enable": "all"
  }
}
# Déplacer manuellement un shard
POST /_cluster/reroute
{
  "commands": [{
    "move": {
      "index": "logs-2024.01.01",
      "shard": 0,
      "from_node": "data-node-1",
      "to_node":   "data-node-5"
    } }] }
```

ISM – Index State Management : cycle de vie des index

ISM automatise les opérations sur les index selon des politiques déclaratives : rollover, passage en read-only, migration vers nœuds cold, suppression après rétention.



Politique ISM complète

```
PUT /_plugins/_ism/policies/logs-lifecycle
{
  "policy": {
    "description": "Hot → Warm → Cold → Delete",
    "default_state": "hot",
    "states": [
      { "name": "hot",
        "actions": [{ "rollover": { "min_index_age": "1d", "min_size": "50gb" } }],
        "transitions": [{ "state_name": "warm", "conditions": { "min_rollover_age": "2d" } }]
      },
      { "name": "warm",
        "actions": [
          { "replica_count": { "number_of_replicas": 1 } },
          { "force_merge": { "max_num_segments": 1 } }
        ],
        "transitions": [{ "state_name": "cold", "conditions": { "min_index_age": "15d" } }]
      },
      { "name": "cold",
        "actions": [
          { "replica_count": { "number_of_replicas": 0 } },
          { "read_only": {} }
        ],
        "transitions": [{ "state_name": "delete", "conditions": { "min_index_age": "90d" } }]
      },
      { "name": "delete", "actions": [{ "delete": {} }], "transitions": [] }
    ],
    "ism_template": [{ "index_patterns": ["logs-*"], "priority": 100 }]
  }
}
```

Gestion du stockage – Architecture Hot-Warm-Cold

L'architecture Hot-Warm-Cold optimise le rapport performance/coût en stockant les données sur des nœuds adaptés à leur fréquence d'accès.

HOT

Nœud : data_hot

Matériel : SSD NVMe, RAM ++

Index en cours (< 1-2 jours)

- Écriture et lecture haute fréquence
- Réplicas actifs
- node.roles: [data_hot]

WARM

Nœud : data_warm

Matériel : SSD ou HDD, RAM moyenne

Index récents, lecture seule (< 30j)

- Forcemerge 1 segment, réplicas 1
- Économie de ressources SSD
- node.roles: [data_warm]

COLD

Nœud : data_cold

Matériel : HDD haute capacité

Archives (> 30 jours)

- read_only, réplicas = 0
- Accès rare, stockage économique
- node.roles: [data_cold]

⚙ Configuration du nœud dans opensearch.yml : node.roles: [data_hot] + node.attr.temperature: hot
→ ISM route via : index.routing.allocation.require.temperature: hot

Bonnes pratiques – Dimensionnement des shards et rollover

Un mauvais dimensionnement des shards est la première source de problèmes de performance. La règle d'or : 10–50 GB par shard, ≤ 20 shards par Go de heap JVM.

Règles de dimensionnement

📦 Taille cible : 10–50 GB par shard

En dessous de 1 GB : overhead trop élevé. Au-dessus de 50 GB : récupération très longue en cas de défaillance.

🔢 Max shards par nœud

≤ 20 shards par Go de heap. Ex : nœud 32 GB heap → max 640 shards (primaires + réplicas).

⚖️ Réplicas selon le contexte

1 réplica en production. 0 pendant une ingestion bulk massive, puis remettre à 1 après.

📋 Rollover plutôt que big shards

Préférer plusieurs petits index via rollover automatique (1 shard/index) à un index monolithique.

Rollover via alias

```
# 1. Créer l'index initial avec alias d'écriture
PUT /logs-000001
{
  "aliases": {
    "logs-write": { "is_write_index": true },
    "logs-read":  {}
  }
}
# 2. Déclencher le rollover (manuel ou via ISM)
POST /logs-write/_rollover
{
  "conditions": {
    "max_age":  "1d",
    "max_size": "40gb",
    "max_docs": 100000000
  }
}
# Résultat : logs-000002 créé automatiquement
# logs-write pointe vers le nouvel index
# logs-read couvre TOUS les index (000001, 000002…)

# Toujours écrire sur l'alias, jamais sur l'index
POST /logs-write/_doc
{ "message": "mon log" }
```

Bonnes pratiques – Index Templates et checklist de maintenance

Les index templates garantissent une configuration cohérente sur tous les index d'un pattern. La maintenance régulière évite l'accumulation de dettes techniques.

Index template composable

```
# 1. Component template : paramètres
PUT /_component_template/logs-settings
{
  "template": {
    "settings": {
      "number_of_shards": 1,
      "number_of_replicas": 1,
      "refresh_interval": "5s",
      "index.search.slowlog.threshold.query.warn": "5s"
    }
  }
}
# 2. Index template composé
PUT /_index_template/logs-template
{
  "index_patterns": ["logs-*"],
  "composed_of": ["logs-settings","logs-mappings"],
  "priority": 200,
  "template": {
    "aliases": { "logs-all": {} }
  }
}
```

Checklist de maintenance

Quotidien

- ✓ cluster health = green ?
- ✓ Shards non assignés ?
- ✓ Heap < 75 % sur tous les nœuds ?

Hebdomadaire

- ✓ Analyser les slow logs
- ✓ Progression des politiques ISM ?
- ✓ Croissance des index nominale ?

Mensuel

- ✓ Revue des politiques ISM/templates
- ✓ Tester la restauration de snapshot
- ✓ Analyse des tendances de performance

Annexes

01 Relations
02 Cache
03 Gestion du stockage

Relations entre documents dans OpenSearch

- **Un moteur de recherche, pas une base relationnelle**
  - OpenSearch est conçu pour stocker des documents JSON autonomes et plats.
  - Les jointures entre documents ne sont pas natives — elles ont un coût.
  - Toute relation doit être modélisée explicitement dans le mapping.
- **2 mécanismes pour représenter des relations**
  - **Nested documents** — objets imbriqués stockés dans le même document Lucene
    - Chaque objet nested est un document Lucene séparé, co-localisé avec le parent
    - Requêtes possibles : nested query, nested aggregation
  - **Relations parent / enfant (join field)** — documents liés par un champ de type join
    - Parent et enfant sont des documents Lucene indépendants dans le même shard
    - Requêtes possibles : has_child, has_parent, parent_id
- **Règle générale**
  - Préférer un modèle de données dénormalisé (document plat) dès que possible.
  - Les relations ne sont à utiliser que lorsque les alternatives ne suffisent pas.

Cardinalité et choix du mapping

- **Relation N-1 : un objet unique par document → object simple**
  - Exemple : un produit appartient à un modèle de produit.
  - Le modèle est un objet unique — pas un tableau — pour chaque produit.
  - → Utiliser object : OpenSearch aplatit en modele.id, modele.nom, modele.marque…
  - → Tout est requêtable normalement, aucun coût supplémentaire.
  - → Le nested est inutile et nuisible dans ce cas.
- **Relation 1-N : un tableau d'objets → nested nécessaire si corrélation inter-champs**
  - Exemple : un produit a plusieurs avis, chaque avis a une note ET un statut vérifié.
  - Sans nested : OpenSearch aplatit en tableaux indépendants.
  - avis.note = [5, 1] et avis.verifie = [true, false]
  - Une requête note>4 ET verifie=true ramène le produit même si ce n'est pas le même avis.
  - La corrélation entre les champs d'un même objet est perdue.
  - Avec nested : la requête garantit que les deux conditions portent sur le même objet du tableau.

Mapping nested — Désavantages

- **Impact sur le volume de données Lucene**
  - Chaque objet nested génère un document Lucene séparé dans le shard.
  - 1 document avec 50 objets nested = 51 documents Lucene.
  - Sur 1 million de documents avec 50 nested chacun → 51 millions de documents Lucene.
  - Impact direct : mémoire, merges, vitesse de recherche.
- **Coût des requêtes**
  - La nested query exécute une sous-requête sur chaque bloc nested, puis effectue un join interne pour remonter au parent.
  - Beaucoup plus lent qu'une simple term query sur un champ plat.
  - L'ordre de grandeur de la dégradation peut atteindre x10 sur de gros volumes.
- **Complexité des agrégations**
  - Toute agrégation sur un champ nested doit être wrappée dans une nested aggregation.
  - Pour remonter au document parent : utiliser *reverse_nested*.
  - Le code devient verbeux et difficile à maintenir.
- **Mise à jour coûteuse**
  - Modifier un objet nested implique de réécrire le document parent entier.
  - Pas de mise à jour partielle d'un objet nested isolé.

Alternatives au relation 1-N

- **1. Objet simple (object) ou nouveau type flattened**
  - OpenSearch aplatit automatiquement les champs : tags.nom, tags.score.
  - Plus simple, plus rapide, aucune requête spéciale nécessaire.
  - Limitation : perd la corrélation entre les champs d'un même objet.
  - Exemple : on ne peut pas chercher tag.nom=rouge ET tag.score>5 sur le MÊME tag.
  - À utiliser quand on n'a pas besoin de cette corrélation inter-champs.
- **2. Créer un champ calculé à l’indexation qui permettra la recherche**
  - Performances optimales, requêtes terms/term simples.
  - Limitations et pas idéal pour des objets « nested » ayant de nombreux attributs

.

- **Question clé avant de choisir nested**

Ai-je une relation 1-N et ai-je besoin de garantir que deux propriétés du même objet sont interrogées ensemble ?
Si non → object ou tableau plat suffisent.

Alternative — Champ calculé

- **Principe — dénormalisation à l'indexation**
  - Créer un champ calculé qui concatène les attributs à corréler en une seule valeur.
  - Aucun nested, aucun join — une simple term query sur un champ keyword.
- **Exemple — produits avec avis (note + vérifié)**
  - Document stocké avec le champ avis_tags calculé :

```
{ "avis": [{"note":5,"verifie":true}, {"note":1,"verifie":false}],
  "avis_tags": ["5_true", "1_false"] }
```

  - Requête : term query sur avis_tags = "5_true"
    → Retrouve exactement les produits avec un avis noté 5 ET vérifié, sans nested query.
- **Calcul avec un ingest pipeline (processor script Painless)**

```
PUT /_ingest/pipeline/avis-tagger
{ "processors": [{ "script": { "lang": "painless", "source": """
    def tags = [];
    for (def a : ctx.avis) { tags.add(a.note + "_" + a.verifie); }
    ctx.avis_tags = tags;
""" }}]}
```

- **Limites**
  - La concaténation doit être pensée à la conception — ajout d'un attribut = réindexation.
  - Ne supporte pas les range queries numériques (note > 4) — pré-calculer des buckets si besoin.
  - Pour corréler beaucoup d'attributs simultanément, le nested reste plus expressif.

Alternative — Relation parent / enfant (join field)

- **Principe**
  - Un seul index contient deux types de documents : parent et enfant.
  - Le champ de type join définit la relation.
  - Parent et enfant sont des documents Lucene indépendants mais doivent être dans le même shard (même routing).
- **Exemple de mapping**
  - mapping : { "relations": { "type": "join", "relations": { "question": "reponse" } } }
  - Document parent : { "titre": "...", "relations": { "name": "question" } }
  - Document enfant : { "corps": "...", "relations": { "name": "reponse", "parent": "ID_PARENT" } }
  - Indexation enfant : PUT /index/_doc/2?routing=ID_PARENT
- **Requêtes disponibles**
  - ***has_child*** — trouve les parents qui ont au moins un enfant correspondant à la query.
  - ***has_parent*** — trouve les enfants dont le parent correspond à la query.
  - ***parent_id*** — trouve tous les enfants d'un parent donné par son ID.

.

Avantages/Inconvénients Relation parent / enfant

- **Avantages vs nested**
  - Les enfants peuvent être mis à jour indépendamment du parent.
  - Pas de réécriture du document parent à chaque modification d'un enfant.
  - Adapté quand les enfants sont nombreux et mis à jour fréquemment.
- **Inconvénients**
  - Requêtes *has_child* et *has_parent* sont très coûteuses (join au moment de la recherche).
  - Le routing doit être géré manuellement à l'indexation.
  - Performances dégradées sur de gros volumes — à éviter en production intensive.

Les types de cache

Node Query · Shard Request · Fielddata · OS Filesystem

Les Types de Cache

01 Node Query Cache

OpenSearch

Met en cache les résultats de clauses filter (context filter uniquement — pas le scoring).

`indices.queries.cache.size → 10% heap JVM`

02 Shard Request Cache

OpenSearch

Met en cache la réponse complète par shard (aggregations, total hits). Actif par défaut uniquement si size: 0.

`indices.requests.cache.size → 1% heap JVM`

03 Fielddata Cache

Lucene / OS

Charge en mémoire les valeurs de champs pour le tri et les aggregations sur des champs text.

`indices.fielddata.cache.size → Illimité (⚠ OOM)`

04 OS Filesystem Cache

Système Linux

Lucene s'appuie sur le page cache Linux pour ses segments. Transparent, géré par l'OS.

`vm.swappiness (sysctl) → Toute la RAM libre`

Invalidation des Caches

Cache	Déclencheur	Conséquence
`Node Query Cache`	Merge de segment Lucene Nouveau document indexé	Cache vidé par shard Reconstitution au prochain accès
`Shard Request Cache`	Refresh de l'index (défaut ~1 s) Ecriture / flush	⚠ **Cause principale de régression entre deux requêtes successives**
`Fielddata Cache`	Eviction LRU (cache plein) close / delete d'index	Rechargement coûteux au prochain tri / aggregation
`OS Filesystem Cache`	Pression mémoire (autre process) Redémarrage du nœud	Relecture disque — impact fort sur la latence des requêtes

💡 Index en lecture seule : désactiver le refresh auto avec "refresh_interval": "-1"

Monitoring des Caches

API _stats

```
# Par index
GET /mon-index/_stats/
  query_cache,request_cache,fielddata

# Tous les nœuds
GET /_nodes/stats/indices/
  query_cache,request_cache

# Forcer le cache (size > 0)
GET /mon-index/_search
  ?request_cache=true
```

Métriques clés

hit_count

Résultats servis depuis le cache

miss_count

Requêtes sans résultat en cache

evictions

Entrées expulsées (cache trop petit ?)

memory_size_in_bytes

Taille occupée dans la heap JVM

Seuils à surveiller

hit_rate < 50% → cache peu efficace, vérifier les invalidations **evictions > 0** → cache trop petit, augmenter indices.requests.cache.size

Profiling — Activer

Ajouter `"profile": true` dans la requête :

```
GET /mon-index/_search
{
  "profile": true,
  "query": {
    "bool": {
      "filter": [
        { "term": { "status": "active" } }
      ],
      "must": [
        { "match": { "title": "opensearch cache" } }
      ]
    }
  }
}
```

Profiling — Réponse (1/2)


```
{
  "profile": {
    "shards": [
      {
        "id": "[node1][mon-index][0]",
        "searches": [
          {
            "query": [
              {
                "type": "BooleanQuery",
                "description": "title:opensearch title:cache #status:active",
                "time_in_nanos": 1842650,
                "breakdown": {
                  "match_count":          42,
                  "build_scorer_count":    1,
                  "create_weight_count":   1,
                  "next_doc_count":       42,
                  "score_count":          42
                },
```

Profiling — Réponse (2/2)

```
                "children": [
                  {
                    "type": "TermQuery",
                    "description": "title:opensearch",
                    "time_in_nanos": 512400
                  },
                  {
                    "type": "TermQuery",
                    "description": "status:active",
                    "time_in_nanos": 98310,
                    "cached": true   // filtre en cache !
                  }
                ]
              }
            ],
            "rewrite_time": 21500,
            "collector": [{
              "name": "SimpleTopScoreDocCollector",
              "time_in_nanos": 310200
            }]
          }
        ]
      }
    ]
  }
}
```

`time_in_nanos`

Temps total sur la clause (cumulé tous segments)

`breakdown`

Détail par opération Lucene : build_scorer, match_count...

`children`

Sous-requêtes de la BooleanQuery avec leur propre timing

`"cached": true`

Filtre servi par le Node Query Cache — pas de traversal segment

`rewrite_time`

Temps de réécriture de la requête avant exécution

Synthèse & Bonnes Pratiques

01	Shard Request Cache invalidé à chaque refresh (~1 s) → principale cause de régression entre deux appels
02	Utiliser size: 0 pour les aggregations pures → bénéficier pleinement du Shard Request Cache
03	Désactiver le refresh auto sur les index en lecture seule : "refresh_interval": "-1"
04	Surveiller hit_count / miss_count via GET /_stats — alerter si evictions > 0
05	Utiliser "profile": true pour identifier les clauses lentes — chercher "cached": true sur les filtres